🐍 Python Scripting for DevOps/SRE/Platform Engineering — Complete Interview Mastery Guide

> **How to use this guide:**
>
> - Work through topics in order, or jump to any section using the Table of Contents
> - ⚠ marks commonly misunderstood or frequently asked topics
> - Every topic includes: concept → internals → interview answers → code → gotchas → DevOps connections
> - After each topic, decide: **Quiz** | **Deeper** | **Next**

📋 TABLE OF CONTENTS — YOUR FULL ROADMAP

🟢 BEGINNER (Foundation)

1. Python Basics for SRE — Data Types, Variables, Operators
2. Control Flow — if/else, loops, comprehensions
3. Functions — def, args, kwargs, decorators basics
4. String Manipulation — format, split, regex basics
5. File I/O — read, write, append, context managers
6. Error Handling — try/except/finally, custom exceptions
7. Modules and Imports — stdlib, pip, virtual environments
8. Data Structures — list, dict, set, tuple, deque

🟡 INTERMEDIATE (Core SRE Skills)

9. os and sys — System Interaction
10. subprocess — Running Shell Commands
11. pathlib — Modern File System Operations
12. logging — Production-grade Logging
13. argparse — CLI Tool Building
14. Environment Variables and Config Management
15. JSON and YAML Parsing
16. HTTP and REST APIs — requests library
17. Concurrency — threading, multiprocessing, asyncio
18. Regular Expressions — re module deep dive

🟠 ADVANCED (Production-Grade)

19. Context Managers — with statement, contextlib
20. Generators and Iterators — memory-efficient pipelines
21. Decorators — Advanced patterns for SRE tooling
22. Dataclasses and Pydantic — Config validation
23. Testing — pytest, mocking, test patterns

24. Package Structure — writing reusable SRE libraries

🔴 EXPERT (Cloud & Infrastructure)

25. AWS SDK — boto3 for automation
26. GCP SDK — google-cloud libraries
27. Kubernetes Automation — kubernetes Python client
28. Docker Automation — docker-py SDK
29. Prometheus and Monitoring Clients
30. CI/CD Scripting Patterns — GitHub Actions, Jenkins
31. Infrastructure as Code with Python — Pulumi, CDK
32. Secret Management — Vault, AWS Secrets Manager
33. Database Automation — psycopg2, SQLAlchemy basics
34. gRPC and Advanced API Patterns

🟢 BEGINNER TOPICS

---

1. Python Basics for SRE

📖 What It Is (Simple Terms)

Python's built-in data types and operators are the atomic building blocks of every script you'll write. For SRE work, you'll constantly be processing strings (log lines, hostnames), numbers (metrics, counts), booleans (health checks), and `None` (missing config values).

🔍 Why It Exists / Problem It Solves

Scripting languages need a type system to represent real-world data — a hostname is different from a port number, which is different from a flag. Python's dynamic typing makes it fast to write automation scripts without boilerplate declarations.

⚙️ How It Works Internally

Python is dynamically typed — every value is an **object** with a type, a reference count, and a value. Variables are just **labels pointing to objects** in memory. This is why `a = b` makes both variables point to the same object, not a copy (matters for mutable types like lists and dicts).

```
a = [1, 2, 3]
b = a           # b points to SAME list object
b.append(4)
print(a)        # [1, 2, 3, 4] — a is also changed!

# To copy:
b = a.copy()    # shallow copy
import copy
b = copy.deepcopy(a)  # deep copy (for nested structures)
```

🧩 Key Components

Type	SRE Use Case	Gotcha
`str`	Log parsing, hostnames, config values	Immutable — every operation creates new string
`int`	Port numbers, exit codes, counts	Python ints are arbitrary precision
`float`	Latency in ms, CPU %, error rates	Floating point precision issues
`bool`	Health check result, feature flags	`True` and `False` are subclasses of `int`
`None`	Missing config, unset env var	`None` is a singleton — use `is None`, not `== None`
`bytes`	Binary data, network payloads	Different from `str` — cannot mix them

```
# SRE-relevant type patterns
hostname = "web-prod-01.us-east-1.internal"
port = 8080
latency_ms = 12.4
is_healthy = True
secret = None  # not yet loaded

# Type checking in scripts
if secret is None:
    raise ValueError("SECRET env var not set")

# ⚠ Common gotcha: truthy/falsy values
if port:       # 0 is falsy, but port 0 is valid! Use explicit check
    pass
if port is not None:  # better
    pass

# String operations SREs use constantly
log_line = "2024-01-15 ERROR web-01 Connection refused to db:5432"
parts = log_line.split()
level = parts[1]          # "ERROR"
host = parts[2]           # "web-01"

# f-strings (Python 3.6+) — use these for readability
service = "api-gateway"
region = "us-east-1"
alert_msg = f"Service {service} in {region} is DOWN"

# Multi-line strings for templates/scripts
bash_cmd = """
#!/bin/bash
kubectl rollout status deployment/{name} -n {namespace}
""".format(name="my-app", namespace="production")
```

💬 Short Interview Answer

"Python's types are all objects, variables are labels not boxes. For SRE scripting the key things to know are: strings are immutable so repeated concatenation in a loop is O(n²) — use join() instead; None should be checked with `is not None` because it's a singleton; and mutable defaults in function signatures are a classic gotcha that can cause state to leak between calls."

🔬 Deep Dive

Python's memory model uses **reference counting** with a **cyclic garbage collector** as backup. The CPython GIL (Global Interpreter Lock) means only one thread executes Python bytecode at a time — this matters when you're writing concurrent monitoring scripts. Every object has `id()` (memory address), `type()`, and a reference count accessible via `sys.getrefcount()`.

```
import sys

x = "hello"
print(sys.getrefcount(x))  # reference count (always >= 2 due to
getrefcount arg)

# Interning: Python caches small integers (-5 to 256) and some strings
a = 256
b = 256
print(a is b)   # True — same object (interned)

a = 257
b = 257
print(a is b)   # False — different objects (NOT interned)
# ⚠ This is why you MUST use == for value comparison, not `is`
```

🏭 Real-World Production Example

```
#!/usr/bin/env python3
"""
parse_healthcheck.py — Parse service health check outputs for alerting
"""
from typing import Optional

def parse_service_status(raw_output: str) -> dict:
    """
    Parse output like: 'web-01 OK 12ms | db-01 FAIL timeout | cache-01 OK
2ms'
    Returns structured data for alerting.
    """
    results = {}
    entries = raw_output.strip().split(" | ")

    for entry in entries:
        parts = entry.split()
        if len(parts) < 2:
            continue
```

```
        name = parts[0]
        status = parts[1]
        latency: Optional[float] = None

        if len(parts) >= 3 and parts[2].endswith("ms"):
            try:
                latency = float(parts[2].rstrip("ms"))
            except ValueError:
                pass

        results[name] = {
            "healthy": status == "OK",
            "status": status,
            "latency_ms": latency
        }

    return results

# Usage
raw = "web-01 OK 12ms | db-01 FAIL timeout | cache-01 OK 2ms"
statuses = parse_service_status(raw)

unhealthy = [name for name, info in statuses.items() if not
info["healthy"]]
if unhealthy:
    print(f"ALERT: {len(unhealthy)} services down: {',
'.join(unhealthy)}")
```

❓ Common Interview Questions

Q: What's the difference between `is` and `==`?

> `==` checks value equality (calls `__eq__`). `is` checks identity — whether two variables point to the same object in memory. Use `is` only for `None`, `True`, `False`, and sentinel values. Never use `is` to compare strings or integers (except in the interned range -5 to 256 where it might work, but it's undefined behavior).

Q: Why is string concatenation in a loop bad?

> Strings are immutable. `s = s + new_part` creates a brand new string object each iteration, copying all previous content. For N iterations of average length L, that's $O(N^2 \cdot L)$ time and memory. Use `"".join(list_of_parts)` instead — $O(N \cdot L)$.

```
# ❌ BAD — O(n²)
result = ""
for line in log_lines:
    result += line + "\n"

# ✅ GOOD — O(n)
result = "\n".join(log_lines) + "\n"
```

Q: What is a mutable default argument bug?

```
# ⚠ CLASSIC BUG — default list is created ONCE at function definition time
def add_host(host, cluster_hosts=[]):
    cluster_hosts.append(host)
    return cluster_hosts

print(add_host("web-01"))   # ["web-01"]
print(add_host("web-02"))   # ["web-01", "web-02"] — BUG! shared state

# ✅ FIX
def add_host(host, cluster_hosts=None):
    if cluster_hosts is None:
        cluster_hosts = []
    cluster_hosts.append(host)
    return cluster_hosts
```

⚠ Tricky Gotchas

- `bool` is a subclass of `int`: `True + True == 2`, `sum([True, False, True]) == 2` (useful for counting health check passes)
- `None` is a singleton but `float('nan') != float('nan')` — NaN comparisons are always False
- Division: `7 / 2 == 3.5` (float), `7 // 2 == 3` (integer floor division)
- `0.1 + 0.2 != 0.3` in floating point — use `math.isclose()` for latency comparisons

🔗 DevOps Connection

Every Terraform output, Kubernetes API response, Prometheus metric, and CloudWatch log you process starts as raw data that must be coerced into Python types. Understanding type coercion, `None` handling, and string immutability prevents subtle bugs in production alerting scripts.

---

2. Control Flow

📖 What It Is (Simple Terms)

Control flow constructs — `if/elif/else`, `for`, `while`, `break`, `continue`, `pass` — let your scripts make decisions and repeat actions. List/dict/set **comprehensions** are Pythonic one-liners for transforming collections.

⚙️ How It Works Internally

Python evaluates truthiness via `__bool__()` (or `__len__()`). The `for` loop calls `iter()` on the object to get an iterator, then repeatedly calls `next()` until `StopIteration` is raised. This is why `for` works on anything iterable — lists, dicts, files, generators, API response streams.

```
# for loop desugared
my_list = [1, 2, 3]
it = iter(my_list)
while True:
    try:
        item = next(it)
        print(item)
    except StopIteration:
        break
```

🧩 Key Patterns for SRE Scripts

```
# ---- CONDITIONALS ----

# Ternary (conditional expression)
status = "healthy" if response_code == 200 else "degraded"

# Walrus operator ⚠ (Python 3.8+) — assign AND test in one line
import re
log_line = "ERROR: disk usage 95% on /dev/sda1"
if match := re.search(r'(\d+)%', log_line):
    usage = int(match.group(1))
    if usage > 90:
        print(f"ALERT: Disk at {usage}%")

# match/case (Python 3.10+) — structural pattern matching
def handle_exit_code(code: int) -> str:
    match code:
        case 0:
            return "success"
        case 1:
            return "general error"
        case 137:
            return "OOMKilled (SIGKILL)"
        case 143:
            return "graceful shutdown (SIGTERM)"
        case _:
            return f"unknown exit code: {code}"

# ---- LOOPS ----

# enumerate — always use when you need index + value
services = ["api", "worker", "scheduler"]
for i, svc in enumerate(services, start=1):
    print(f"{i}/{len(services)}: checking {svc}")

# zip — iterate multiple lists together
hosts = ["web-01", "web-02", "web-03"]
statuses = ["healthy", "degraded", "healthy"]
for host, status in zip(hosts, statuses):
```

```
    if status != "healthy":
        print(f"ALERT: {host} is {status}")

# zip_longest — when lists may have different lengths
from itertools import zip_longest
for host, status in zip_longest(hosts, statuses, fillvalue="unknown"):
    print(host, status)

# dict iteration patterns
config = {"timeout": 30, "retries": 3, "region": "us-east-1"}
for key, value in config.items():
    print(f"{key}: {value}")

# ---- COMPREHENSIONS ----

# List comprehension — filter + transform
log_lines = ["INFO: ok", "ERROR: disk full", "WARN: high mem", "ERROR:
oom"]
errors = [line for line in log_lines if line.startswith("ERROR")]

# Dict comprehension — restructure data
raw_metrics = [("cpu", 72.4), ("mem", 88.1), ("disk", 45.2)]
metrics_dict = {name: value for name, value in raw_metrics}

# Set comprehension — unique values
all_levels = {line.split(":")[0] for line in log_lines}  # {'INFO',
'ERROR', 'WARN'}

# Nested comprehension (flatten pod list from multiple nodes)
node_pods = {
    "node-01": ["pod-a", "pod-b"],
    "node-02": ["pod-c"],
    "node-03": ["pod-d", "pod-e", "pod-f"],
}
all_pods = [pod for node_pods_list in node_pods.values() for pod in
node_pods_list]

# Generator expression (memory efficient — doesn't build list in memory)
total_errors = sum(1 for line in log_lines if "ERROR" in line)

# ---- LOOP CONTROL ----

# break — stop loop
for attempt in range(1, 6):
    response = check_health("https://api.example.com/health")
    if response.status_code == 200:
        print(f"Healthy after {attempt} attempt(s)")
        break
else:
    # for-else: else block runs only if loop completed WITHOUT break
    print("Service never became healthy after 5 attempts — paging on-
call")

# continue — skip to next iteration
```

```
for log in log_lines:
    if log.startswith("DEBUG"):
        continue  # skip debug logs
    process(log)
```

💬 Short Interview Answer

"For SRE scripting, comprehensions are critical for clean, readable data transformation — I use list comprehensions for filtering logs, dict comprehensions for restructuring API responses, and generator expressions when processing large files to avoid loading everything into memory. The for-else construct is underused but perfect for 'retry until success or exhaust attempts' patterns. The walrus operator in Python 3.8+ is great for parsing log lines where you test a regex and need the match object."

🏭 Real-World Production Example

```
#!/usr/bin/env python3
"""
find_oom_pods.py — Scan kubectl describe output to find OOMKilled pods
"""
import subprocess
import json

def get_oom_killed_pods(namespace: str = "production") -> list[dict]:
    """Find all OOMKilled pods in a namespace."""
    result = subprocess.run(
        ["kubectl", "get", "pods", "-n", namespace, "-o", "json"],
        capture_output=True, text=True, check=True
    )
    data = json.loads(result.stdout)

    oom_pods = []
    for pod in data.get("items", []):
        pod_name = pod["metadata"]["name"]

        # Check all containers' last termination state
        containers = pod.get("status", {}).get("containerStatuses", [])
        for container in containers:
            last_state = container.get("lastState", {}).get("terminated",
{})
            if last_state.get("reason") == "OOMKilled":
                oom_pods.append({
                    "pod": pod_name,
                    "container": container["name"],
                    "exit_code": last_state.get("exitCode"),
                    "finished_at": last_state.get("finishedAt"),
                })

    return oom_pods

oom_pods = get_oom_killed_pods()
if oom_pods:
```

```
    print(f"Found {len(oom_pods)} OOMKilled container(s):")
    for p in oom_pods:
        print(f"  {p['pod']}/{p['container']} at {p['finished_at']}")
else:
    print("No OOMKilled pods found")
```

⚠ Tricky Gotchas

- **Modifying a list while iterating it** is undefined behavior — use `[x for x in lst if condition]` instead
- **`for-else`** — the `else` runs when the loop finishes normally (no `break`), NOT when the iterable is empty
- `range(len(lst))` — almost always wrong in Python; use `enumerate(lst)` instead
- Dict insertion order is preserved in Python 3.7+ — but don't rely on this for sorted output; use `sorted()`

🔗 DevOps Connection

Comprehensions are how you transform raw API responses (Kubernetes pod lists, AWS instance arrays, Prometheus query results) into actionable data structures efficiently. The `for-else` pattern maps directly to "retry with timeout" logic in health check scripts.

---

3. Functions

📖 What It Is (Simple Terms)

Functions are reusable, named blocks of code. Python functions are **first-class objects** — they can be assigned to variables, passed as arguments, and returned from other functions. This is what makes decorators, callbacks, and higher-order functions work.

⚙️ How It Works Internally

When Python calls a function, it creates a new **frame** (stack frame) with a local namespace. Arguments are bound to parameter names in this local scope. Closures capture variables from the enclosing scope by **reference**, not by value — a classic gotcha.

```
# Closure captures by reference, not value
def make_checkers(services):
    checkers = []
    for svc in services:
        # ⚠ BUG: all closures capture the SAME variable `svc`
        checkers.append(lambda: print(f"checking {svc}"))
    return checkers

fns = make_checkers(["api", "db", "cache"])
for fn in fns:
    fn()
# Output: checking cache / checking cache / checking cache — BUG!
```

```
# ✅ FIX: default argument to capture value at definition time
def make_checkers(services):
    checkers = []
    for svc in services:
        checkers.append(lambda s=svc: print(f"checking {s}"))
    return checkers
```

🧩 Key Components

```
# ---- ARGUMENT TYPES ----

def deploy(
    service: str,            # positional
    version: str,            # positional
    namespace: str = "default",  # keyword with default
    dry_run: bool = False,   # keyword with default
    *extra_args,             # variadic positional (*args)
    **labels                 # variadic keyword (**kwargs)
) -> dict:
    """Deploy a service to Kubernetes."""
    print(f"Deploying {service}:{version} to {namespace}")
    print(f"Extra args: {extra_args}")
    print(f"Labels: {labels}")
    return {"status": "deployed", "dry_run": dry_run}

# Call examples
deploy("api-gateway", "v1.2.3")
deploy("api-gateway", "v1.2.3", namespace="production", dry_run=True)
deploy("api-gateway", "v1.2.3", "extra", team="platform", env="prod")

# Keyword-only arguments (after *)
def restart_pod(pod_name: str, *, namespace: str, force: bool = False):
    """namespace MUST be passed as keyword arg"""
    pass

restart_pod("my-pod", namespace="prod")      # ✅
# restart_pod("my-pod", "prod")             # ❌ TypeError

# ---- TYPE HINTS ----
from typing import Optional, Union, List, Dict, Callable, Any

def check_services(
    hosts: list[str],
    timeout: Optional[int] = None,
    callback: Optional[Callable[[str, bool], None]] = None
) -> dict[str, bool]:
    results = {}
    for host in hosts:
        healthy = ping(host, timeout=timeout or 5)
        results[host] = healthy
```

```
        if callback:
            callback(host, healthy)
    return results

# ---- LAMBDA (anonymous functions) ----
services = [{"name": "api", "priority": 1}, {"name": "db", "priority": 3}]
sorted_by_priority = sorted(services, key=lambda s: s["priority"],
reverse=True)

# ---- HIGHER ORDER FUNCTIONS ----
from functools import partial, reduce

# partial — pre-fill some arguments
def restart_service(service, env, timeout=30):
    print(f"Restarting {service} in {env} with timeout {timeout}s")

restart_in_prod = partial(restart_service, env="production")
restart_in_prod("api-gateway")  # reuses env="production"

# map and filter — often replaced by comprehensions but useful with
callables
pod_names = ["web-abc123", "api-def456", "db-ghi789"]
service_names = list(map(lambda p: p.rsplit("-", 1)[0], pod_names))
# ["web", "api", "db"]

# reduce — fold a list to a single value
from functools import reduce
total_replicas = reduce(lambda acc, svc: acc + svc["replicas"], services,
0)

# ---- DOCSTRINGS ----
def rollback_deployment(name: str, namespace: str, version: str) -> bool:
    """
    Roll back a Kubernetes deployment to a specific version.

    Args:
        name: Deployment name (e.g., 'api-gateway')
        namespace: Kubernetes namespace
        version: Target image tag to roll back to

    Returns:
        True if rollback succeeded, False otherwise

    Raises:
        subprocess.CalledProcessError: If kubectl command fails
        ValueError: If version string is empty

    Example:
        >>> rollback_deployment("api", "production", "v1.2.2")
        True
    """
    if not version:
        raise ValueError("version cannot be empty")
    # ... implementation
```

💬 Short Interview Answer

*"Python functions are first-class objects, which enables patterns like decorators, callbacks, and dependency injection that are essential in SRE tooling. The key argument types to understand are positional, keyword-with-default, *args for variadic positional, **kwargs for variadic keyword, and keyword-only (after bare *). The main gotchas are mutable defaults — always use None and initialize inside the function body — and closure capture by reference, which bites you in loops."*

🏭 Real-World Production Example

```
#!/usr/bin/env python3
"""
retry_with_backoff.py — Reusable retry utility for SRE scripts
"""
import time
import logging
from typing import Callable, TypeVar, Optional, Type
from functools import wraps

T = TypeVar("T")
logger = logging.getLogger(__name__)

def retry(
    max_attempts: int = 3,
    backoff_seconds: float = 1.0,
    backoff_factor: float = 2.0,
    exceptions: tuple[Type[Exception], ...] = (Exception,),
    on_retry: Optional[Callable[[int, Exception], None]] = None,
) -> Callable:
    """
    Decorator factory that retries a function with exponential backoff.

    Usage:
        @retry(max_attempts=5, backoff_seconds=2, exceptions=
(requests.Timeout,))
        def call_external_api(url: str) -> dict:
            ...
    """
    def decorator(func: Callable) -> Callable:
        @wraps(func)  # preserves func.__name__, __doc__, etc.
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_exception = e
                    if attempt == max_attempts:
                        logger.error(
                            f"{func.__name__} failed after {max_attempts}
```

```
attempts: {e}"
                        )
                        raise

                    wait = backoff_seconds * (backoff_factor ** (attempt -
1))
                    logger.warning(
                        f"{func.__name__} attempt {attempt} failed: {e}. "
                        f"Retrying in {wait:.1f}s..."
                    )
                    if on_retry:
                        on_retry(attempt, e)
                    time.sleep(wait)
            raise last_exception  # unreachable but satisfies type checker
        return wrapper
    return decorator

# Usage
import requests

@retry(max_attempts=5, backoff_seconds=1, exceptions=
(requests.RequestException,))
def fetch_metrics(prometheus_url: str, query: str) -> dict:
    resp = requests.get(
        f"{prometheus_url}/api/v1/query",
        params={"query": query},
        timeout=10
    )
    resp.raise_for_status()
    return resp.json()

data = fetch_metrics("http://prometheus:9090", "up{job='api-gateway'}")
```

⚠ Tricky Gotchas

- **`@wraps(func)` is mandatory in decorators** — without it, `func.__name__` and `func.__doc__` are lost, breaking introspection, logging, and pytest
- **Default arguments evaluated at definition time** — `def f(t=time.time())` captures time ONCE
- **`*args` is a tuple, `**kwargs` is a dict** — not lists/dicts
- Unpacking into function calls: `f(*list_of_args, **dict_of_kwargs)` — essential for forwarding arguments

🔗 DevOps Connection

The retry decorator pattern shown above is the backbone of every resilient SRE script that talks to flaky external APIs (Kubernetes API server, cloud provider APIs, Prometheus). Understanding decorators lets you add cross-cutting concerns (retry, timeout, circuit breaking, audit logging) without modifying business logic.

---

4. String Manipulation

📖 What It Is (Simple Terms)

Python's string operations — slicing, splitting, formatting, searching, replacing — are used constantly in SRE work for parsing log lines, building API URLs, formatting alert messages, and processing config files.

⚙️ How It Works Internally

Strings in Python 3 are Unicode by default (UTF-8 internally represented as either Latin-1, UCS-2, or UCS-4 depending on content, to save memory). String operations mostly return new string objects (immutable). The `re` module compiles regex patterns to finite automata.

🧩 Key Patterns

```
# ---- FORMATTING ----

# f-strings (Python 3.6+) — preferred for readability
service = "api-gateway"
latency = 145.7
f"Service {service} responded in {latency:.2f}ms"  # "...in 145.70ms"
f"{'ERROR':>10}"     # right-align in 10 chars: "     ERROR"
f"{1024 * 1024:,}"   # thousands separator: "1,048,576"

# .format() — useful when template is a variable (not literal)
template = "Host {host} is {status}"
msg = template.format(host="web-01", status="DOWN")

# % formatting — legacy, avoid in new code
# f-string with dict unpacking
data = {"host": "web-01", "region": "us-east-1"}
msg = "Alert: {host} in {region}".format(**data)

# ---- SPLITTING AND JOINING ----

# split with maxsplit — important for log parsing
log = "2024-01-15T10:23:45Z ERROR web-01 Connection refused: db:5432"
timestamp, level, host, *message = log.split(None, 3)  # None = any
whitespace
print(message)  # ['Connection refused: db:5432']

# rsplit — split from right (useful for paths)
path = "/apps/production/api-gateway/config.yaml"
dir_path, filename = path.rsplit("/", 1)

# partition — split into exactly 3 parts around first occurrence
key, sep, value = "timeout=30".partition("=")
# "timeout", "=", "30"

# join — the RIGHT way to build strings from lists
hosts = ["web-01", "web-02", "web-03"]
print(", ".join(hosts))       # "web-01, web-02, web-03"
print("\n".join(hosts))       # multi-line
ansible_hosts = "\n".join(f"{h} ansible_user=ubuntu" for h in hosts)
```

```python
# ---- SEARCHING AND TESTING ----

log_line = "  ERROR: disk usage 95% on /dev/sda1  "

# strip/lstrip/rstrip
clean = log_line.strip()  # removes leading/trailing whitespace

# startswith/endswith accept tuples
if clean.startswith(("ERROR", "CRITICAL", "FATAL")):
    page_oncall(clean)

# in operator for substring check
if "OOMKilled" in pod_status:
    print("Pod was OOM killed")

# find vs index
idx = clean.find("disk")   # returns -1 if not found
idx = clean.index("disk")  # raises ValueError if not found

# ---- REPLACE AND TRANSFORM ----

# replace
sanitized = log_line.replace("password=secret123", "password=***")

# translate - for bulk character replacement
table = str.maketrans(".-/", "___")  # replace . - / with _
safe_name = "us-east-1.prod/api".translate(table)  # "us_east_1_prod_api"

# case methods
"CRITICAL".lower()   # "critical"
"warning".upper()    # "WARNING"
"hello world".title() # "Hello World"

# ---- REGEX (re module) ----
import re

# Compile pattern for reuse (much faster in loops)
LOG_PATTERN = re.compile(
    r'^(?P<timestamp>\d{4}-\d{2}-\d{2}T[\d:Z]+)\s+'
    r'(?P<level>ERROR|WARN|INFO|DEBUG)\s+'
    r'(?P<host>[\w-]+)\s+'
    r'(?P<message>.+)$'
)

def parse_log_line(line: str) -> dict | None:
    match = LOG_PATTERN.match(line.strip())
    if not match:
        return None
    return match.groupdict()

# Named groups make code self-documenting
result = parse_log_line("2024-01-15T10:23:45Z ERROR web-01 Connection
refused")
```

```
# {'timestamp': '2024-01-15T10:23:45Z', 'level': 'ERROR', 'host': 'web-
01', ...}

# re.findall vs re.finditer (finditer is memory-efficient for large texts)
text = "Pods: web-abc123 api-def456 db-ghi789"
pod_ids = re.findall(r'[a-z]+-[a-z0-9]+', text)

# Substitution with function
def mask_ip(match):
    parts = match.group().split(".")
    return f"{parts[0]}.{parts[1]}.xxx.xxx"

log = "Connection from 192.168.1.100 to 10.0.0.5"
masked = re.sub(r'\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b', mask_ip, log)

# ---- MULTILINE AND TEMPLATES ----
from string import Template

# Safe templating (won't raise on missing keys with safe_substitute)
tmpl = Template("Alert: $service is $status in $region")
msg = tmpl.safe_substitute(service="api", status="DOWN")
# "Alert: api is DOWN in $region" - $region left unchanged

# textwrap for formatting CLI output
import textwrap
long_message = "This is a very long alert message that needs to be wrapped
properly for terminal display."
print(textwrap.fill(long_message, width=80))
print(textwrap.indent(long_message, prefix="  "))  # add indentation
```

💬 Short Interview Answer

"String manipulation is everywhere in SRE work — log parsing, URL building, config templating. Key things: always use f-strings for readability, compile regex patterns with `re.compile()` when used in loops (10x+ speedup), use `.partition()` instead of `.split('=', 1)` for key=value parsing, and `''.join()` instead of `+=` for building strings. For log parsing with complex formats, named groups in regex make the code self-documenting."

⚠ Tricky Gotchas

- `str.split()` with no arguments splits on ANY whitespace and removes empty strings — `str.split(' ')` with explicit space does NOT
- `re.match()` only matches at the start of string; `re.search()` searches anywhere — use `re.fullmatch()` to match entire string
- Regex backslashes: use raw strings `r'\d+'` to avoid `\\d+`
- `.strip()` removes ALL leading/trailing whitespace, including `\n` — use `.rstrip('\n')` if you only want to remove newlines

🔗 DevOps Connection

Log parsing, alert message formatting, config file generation, Ansible inventory building, Kubernetes manifest templating — all string manipulation. Compiled regex patterns are critical in log processors that parse millions of lines per minute.

---

5. File I/O

📖 What It Is (Simple Terms)

Reading and writing files — configuration files, log files, lock files, PID files, JSON/YAML configs, SSH keys — is fundamental SRE automation. Python's `open()` built-in plus context managers make this safe and reliable.

⚙️ How It Works Internally

`open()` returns a file object that wraps an OS file descriptor. The **context manager** (`with` statement) calls `__enter__()` on open and `__exit__()` on close (even if an exception occurs). Buffered I/O means data may not be written to disk immediately — use `flush()` or `fsync()` for critical writes.

🧩 Key Patterns

```
import os
import json
import csv
import tempfile
import shutil
from pathlib import Path

# ---- BASIC READ/WRITE ----

# ALWAYS use context managers — guaranteed file close
with open("/etc/hosts", "r", encoding="utf-8") as f:
    content = f.read()  # entire file as string

# Line by line (memory efficient for large files)
with open("/var/log/app.log", "r") as f:
    for line in f:  # file object IS an iterator
        if "ERROR" in line:
            print(line.rstrip())

# Read all lines into list
with open("config.txt", "r") as f:
    lines = f.readlines()  # includes \n
    lines = [l.strip() for l in f.readlines()]  # stripped

# Write (creates or truncates)
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello World\n")
    f.writelines(["line1\n", "line2\n"])  # no separator added

# Append
```

```python
with open("audit.log", "a") as f:
    f.write(f"{timestamp}: {user} deployed {service}\n")

# ---- FILE MODES ----
# r   read (default)
# w   write (truncate)
# a   append
# x   exclusive create (fails if exists) — for lock files!
# b   binary mode suffix: rb, wb
# +   read+write: r+, w+

# Exclusive create for lock files
try:
    with open("/tmp/deploy.lock", "x") as f:
        f.write(str(os.getpid()))
    # ... do deployment
finally:
    os.unlink("/tmp/deploy.lock")

# Binary mode for executables, images, certificates
with open("cert.pem", "rb") as f:
    cert_bytes = f.read()

# ---- SEEK AND TELL ----
with open("big_file.log", "r") as f:
    f.seek(0, 2)  # seek to end
    size = f.tell()  # get file size
    f.seek(max(0, size - 1024))  # read last 1KB
    tail = f.read()

# ---- ATOMIC WRITES ----
# ⚠ Never write directly to production config — use atomic write pattern
def atomic_write(filepath: str, content: str) -> None:
    """Write content atomically by writing to temp file then renaming."""
    path = Path(filepath)
    tmp_path = path.with_suffix(".tmp")
    try:
        with open(tmp_path, "w", encoding="utf-8") as f:
            f.write(content)
            f.flush()
            os.fsync(f.fileno())  # ensure bytes hit disk
        os.rename(tmp_path, filepath)  # atomic on POSIX
    except Exception:
        tmp_path.unlink(missing_ok=True)
        raise

# ---- TEMPFILE (safe temporary files) ----
# NamedTemporaryFile — deleted when closed
with tempfile.NamedTemporaryFile(mode='w', suffix='.yaml', delete=False)
as f:
    f.write(yaml_content)
    tmp_path = f.name
# Use tmp_path, then clean up
os.unlink(tmp_path)
```

```
# TemporaryDirectory — auto-cleaned directory
with tempfile.TemporaryDirectory() as tmpdir:
    manifest_path = os.path.join(tmpdir, "deployment.yaml")
    # write manifest, apply with kubectl, tmpdir auto-deleted

# ---- JSON FILE I/O ----
# Read
with open("config.json", "r") as f:
    config = json.load(f)  # parses JSON from file object

# Write (pretty-printed, sorted keys for diffability)
with open("config.json", "w") as f:
    json.dump(config, f, indent=2, sort_keys=True, default=str)
    # default=str handles non-serializable types (datetime, Decimal, etc.)

# ---- CSV FILE I/O ----
# Read CSV with DictReader
with open("hosts.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    hosts = list(reader)  # [{'hostname': 'web-01', 'ip': '10.0.0.1',
...}, ...]

# Write CSV
fieldnames = ["hostname", "ip", "region", "status"]
with open("inventory.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    for host in hosts:
        writer.writerow(host)

# ---- FILE OPERATIONS ----
# shutil for higher-level operations
shutil.copy("app.conf", "app.conf.bak")         # copy file
shutil.copytree("configs/", "configs.bak/")      # copy directory tree
shutil.move("old_path/file", "new_path/file")     # move/rename
shutil.rmtree("old_directory/", ignore_errors=True) # remove directory
tree
shutil.make_archive("backup_2024", "gztar", "configs/") # create tar.gz

# ---- READING LAST N LINES (tail -n equivalent) ----
from collections import deque

def tail(filepath: str, n: int = 100) -> list[str]:
    """Memory-efficient tail: reads only last n lines."""
    with open(filepath, "r") as f:
        return list(deque(f, maxlen=n))

# ---- WATCHING FILES ----
import time
def follow_log(filepath: str):
    """Like 'tail -f' — follow a log file."""
    with open(filepath, "r") as f:
        f.seek(0, 2)  # start at end
```

```
        while True:
            line = f.readline()
            if line:
                yield line.rstrip()
            else:
                time.sleep(0.1)  # wait for more data
```

💬 Short Interview Answer

"File I/O in Python should always use context managers — the `with` statement guarantees file closure even on exceptions. For production scripts, I use the atomic write pattern: write to a `.tmp` file, `fsync()` to flush to disk, then `os.rename()` which is atomic on POSIX systems. For large log files, iterate line by line rather than reading the whole file into memory. For lock files, use exclusive create mode `'x'` which atomically fails if the file exists."

⚠️ Tricky Gotchas

- **Forgetting `newline=""` in CSV mode** — without it on Windows, you get double line breaks
- **`f.write()` doesn't add newlines** — you must include `\n` explicitly
- **`os.rename()` is atomic only on the same filesystem** — across filesystems, use `shutil.move()` but know it's not atomic
- **`json.dump()` vs `json.dumps()`** — `dump` writes to file, `dumps` returns string; same pattern for `load`/`loads`
- Large file encoding: always specify `encoding="utf-8"` explicitly — default varies by OS

🔗 DevOps Connection

Config file management (Nginx, Kubernetes manifests, Ansible vars), log rotation scripts, deployment artifact management, SSH key provisioning, audit logging — all require robust file I/O with atomic writes and proper error handling.

---

6. Error Handling

📖 What It Is (Simple Terms)

`try/except/else/finally` blocks let your scripts handle failures gracefully — catch specific exceptions, log them, retry, or fail with a meaningful message instead of a raw stack trace. Custom exceptions make your SRE tools self-documenting.

⚙️ How It Works Internally

When an exception is raised, Python unwinds the call stack looking for a matching `except` clause. The exception object carries a traceback (the stack of frames). The `finally` block ALWAYS runs — even if there's a `return` inside the `try` block. Exception chaining with `raise X from Y` preserves the original cause.

🧩 Key Patterns

```python
import sys
import traceback
import logging
from typing import Optional

logger = logging.getLogger(__name__)

# ---- EXCEPTION HIERARCHY ----
# BaseException
#   SystemExit        (sys.exit())
#   KeyboardInterrupt (Ctrl+C)
#   Exception         (catch-all for normal errors)
#     OSError         (IOError, FileNotFoundError, PermissionError, ...)
#     ValueError      (wrong type/value)
#     KeyError        (missing dict key)
#     IndexError      (list out of bounds)
#     TypeError       (wrong type)
#     AttributeError  (missing attribute)
#     RuntimeError    (general runtime error)
#     subprocess.CalledProcessError
#     requests.RequestException

# ---- BASIC STRUCTURE ----
try:
    result = risky_operation()
except SpecificError as e:
    # Handle specific case
    logger.error(f"Specific error: {e}")
except (TypeError, ValueError) as e:
    # Handle multiple exception types
    logger.error(f"Type/value error: {e}")
except Exception as e:
    # Catch-all (last resort)
    logger.exception("Unexpected error")  # logs with full traceback
    raise  # re-raise to propagate
else:
    # Runs only if NO exception was raised in try
    logger.info("Operation succeeded")
finally:
    # ALWAYS runs (cleanup code goes here)
    cleanup_resources()

# ---- CUSTOM EXCEPTIONS ----
class DeploymentError(Exception):
    """Base exception for deployment failures."""
    pass

class RolloutTimeoutError(DeploymentError):
    """Raised when a rollout exceeds the timeout."""
    def __init__(self, deployment: str, timeout: int, current_replicas:
int):
        self.deployment = deployment
        self.timeout = timeout
```

```
        self.current_replicas = current_replicas
        super().__init__(
            f"Deployment '{deployment}' did not complete within {timeout}s
"
            f"(only {current_replicas} replicas ready)"
        )

class HealthCheckError(DeploymentError):
    """Raised when post-deployment health check fails."""
    def __init__(self, url: str, status_code: int):
        self.url = url
        self.status_code = status_code
        super().__init__(f"Health check failed: {url} returned
{status_code}")

# Usage
try:
    deploy_and_verify("api-gateway", "v1.2.3")
except RolloutTimeoutError as e:
    logger.error(f"Rollout timed out: {e}")
    rollback(e.deployment)
    sys.exit(1)
except HealthCheckError as e:
    logger.error(f"Health check failed for {e.url}: HTTP {e.status_code}")
    sys.exit(2)
except DeploymentError as e:
    logger.error(f"Deployment failed: {e}")
    sys.exit(3)

# ---- EXCEPTION CHAINING ----
def load_config(path: str) -> dict:
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError as e:
        raise ConfigError(f"Config file not found: {path}") from e
    except json.JSONDecodeError as e:
        raise ConfigError(f"Invalid JSON in config: {path}") from e

# ---- CONTEXT: SUPPRESS SPECIFIC EXCEPTIONS ----
from contextlib import suppress

# Clean way to ignore specific exceptions
with suppress(FileNotFoundError):
    os.unlink("/tmp/stale_lock_file")

# ---- EXCEPTION GROUPS (Python 3.11+) ----
# For handling multiple concurrent errors (e.g., asyncio.TaskGroup)
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(check_service("api"))
        tg.create_task(check_service("db"))
except* ConnectionError as eg:
    for e in eg.exceptions:
```

```
        logger.error(f"Connection error: {e}")

# ---- LOGGING EXCEPTIONS PROPERLY ----
try:
    result = call_api()
except Exception as e:
    # ❌ BAD — loses traceback
    logger.error(f"Error: {e}")

    # ✅ GOOD — full traceback in log
    logger.exception("API call failed")

    # ✅ ALSO GOOD — manual traceback string
    logger.error("API call failed", exc_info=True)

    # Get traceback as string (for sending to Slack/PagerDuty)
    tb = traceback.format_exc()
    notify_oncall(f"Script failed:\n```{tb}```")

# ---- ASSERT (only for development, NOT production) ----
# ⚠ assert statements can be disabled with python -O flag
assert config.get("region"), "region must be set"  # BAD for production
if not config.get("region"):
    raise ValueError("region must be set")  # GOOD for production
```

💬 Short Interview Answer

"Good exception handling in SRE scripts means: catch specific exceptions, not bare `except Exception` unless you're at the top level of a script. Use `logger.exception()` which automatically includes the full traceback. Create custom exception classes that carry context — like the deployment name, timeout value, and current replica count — so that alert messages and runbooks have actionable data. Use `raise X from Y` for exception chaining so the original cause isn't lost when you wrap exceptions."

🏭 Real-World Production Example

```
#!/usr/bin/env python3
"""
deploy_with_verification.py — Safe deployment with rollback on failure
"""
import sys
import logging
import subprocess
import time

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s")
logger = logging.getLogger(__name__)

class DeploymentError(Exception):
    pass
```

```
def run_kubectl(*args, check=True) -> subprocess.CompletedProcess:
    """Run kubectl command with proper error handling."""
    cmd = ["kubectl"] + list(args)
    logger.debug(f"Running: {' '.join(cmd)}")
    try:
        return subprocess.run(cmd, capture_output=True, text=True,
check=check)
    except subprocess.CalledProcessError as e:
        raise DeploymentError(
            f"kubectl command failed: {' '.join(cmd)}\n"
            f"stdout: {e.stdout}\nstderr: {e.stderr}"
        ) from e
    except FileNotFoundError:
        raise DeploymentError("kubectl not found — is it installed and in
PATH?")

def deploy(name: str, image: str, namespace: str = "default") -> None:
    logger.info(f"Deploying {name} with image {image}")
    try:
        run_kubectl("set", "image", f"deployment/{name}", f"{name}=
{image}", "-n", namespace)

        logger.info("Waiting for rollout...")
        result = run_kubectl(
            "rollout", "status", f"deployment/{name}",
            "-n", namespace, "--timeout=5m", check=False
        )
        if result.returncode != 0:
            raise DeploymentError(
                f"Rollout failed or timed
out:\n{result.stdout}\n{result.stderr}"
            )
        logger.info("Rollout complete!")

    except DeploymentError:
        logger.error("Deployment failed — initiating rollback")
        try:
            run_kubectl("rollout", "undo", f"deployment/{name}", "-n",
namespace)
            logger.info("Rollback complete")
        except DeploymentError as rollback_err:
            logger.critical(f"ROLLBACK ALSO FAILED: {rollback_err}")
        raise  # re-raise original error

if __name__ == "__main__":
    try:
        deploy("api-gateway", "my-registry/api-gateway:v1.2.3",
"production")
        sys.exit(0)
    except DeploymentError as e:
        logger.error(f"Deployment failed: {e}")
        sys.exit(1)
    except KeyboardInterrupt:
```

```
        logger.warning("Interrupted by user")
        sys.exit(130)  # convention: 128 + signal number (SIGINT=2)
```

⚠ Tricky Gotchas

- **`except Exception` catches `SystemExit` and `KeyboardInterrupt`?** NO — those inherit from `BaseException`, not `Exception`. But be careful with bare `except:` which catches everything.
- **`finally` runs even after `return`** — a `return` in `finally` overrides the `try`'s return value
- **Catching `KeyboardInterrupt`** — if you catch it, always re-raise it or call `sys.exit(130)` by convention
- **`assert` is disabled by `python -O`** — never use it for input validation in production

🔗 DevOps Connection

Deployment scripts MUST handle failures gracefully with automatic rollback. CI/CD pipelines need proper exit codes (0 = success, non-zero = failure) so pipeline tools know whether to proceed. Custom exception classes make post-mortems easier — the exception message itself serves as an incident report.

---

7. Modules and Imports

📖 What It Is (Simple Terms)

Python's module system lets you organize code into files (modules) and packages (directories). The standard library provides modules for virtually every SRE task — OS interaction, networking, cryptography, JSON, CSV. `pip` manages third-party packages. Virtual environments isolate project dependencies.

⚙️ How It Works Internally

When you `import foo`, Python searches `sys.path` for `foo.py` or a `foo/` directory with `__init__.py`. The first time a module is imported, Python compiles it to bytecode (`.pyc`) and caches the module object in `sys.modules`. Subsequent imports just return the cached object — **import is idempotent**.

```
import sys

# sys.path is searched in order
print(sys.path)
# ['', '/usr/lib/python3.11', '/usr/local/lib/python3.11/dist-packages',
...]

# sys.modules is the import cache
import json
print(json is sys.modules['json'])  # True — same object

# Force reimport (rarely needed, mostly for REPL debugging)
import importlib
importlib.reload(my_module)
```

🧩 Key Patterns

```python
# ---- IMPORT STYLES ----

# Standard import
import os
import sys
import subprocess
import json
import logging
from pathlib import Path
from typing import Optional, Union, List
from datetime import datetime, timedelta
from collections import defaultdict, Counter, deque
import time
import re
import hashlib
import base64
import hmac
import ssl
import socket
import threading
import multiprocessing
from functools import wraps, partial, lru_cache
from contextlib import contextmanager, suppress
from dataclasses import dataclass, field
import tempfile
import shutil
import glob
import fnmatch
import itertools
import functools
import operator
import copy
import abc
import enum
import uuid
import struct
import io
import csv
import configparser
import argparse
import textwrap
import traceback
import inspect
import ast
import math
import statistics
import random
import heapq
import bisect
import weakref
```

```python
import gc
import atexit
import signal
import platform
import getpass
import pwd
import grp
import stat
import urllib.parse
import urllib.request
import http.client
import smtplib
import email.mime.text
import zipfile
import tarfile
import gzip
import hashlib
import secrets
import hmac

# Aliased imports
import numpy as np          # numeric computing (less common in SRE)
import yaml                 # PyYAML - YAML parsing (pip install pyyaml)
import requests             # HTTP (pip install requests)
import boto3                # AWS SDK (pip install boto3)
import kubernetes           # k8s client (pip install kubernetes)

# From imports (specific items)
from os import environ, getcwd, listdir
from os.path import join, exists, dirname, basename, abspath

# ---- LAZY IMPORTS (for faster startup) ----
def get_boto3():
    """Import boto3 only when needed (heavy module)."""
    import boto3
    return boto3

# ---- CONDITIONAL IMPORTS ----
try:
    import ujson as json  # faster JSON library if available
except ImportError:
    import json  # fallback to stdlib

# ---- __all__ - control what 'from module import *' exports ----
# In your module file:
__all__ = ['deploy', 'rollback', 'DeploymentError']

# ---- VIRTUAL ENVIRONMENTS ----
# Create
# python -m venv .venv

# Activate
# source .venv/bin/activate    (Linux/Mac)
# .venv\Scripts\activate.bat   (Windows)
```

```python
# Install dependencies
# pip install -r requirements.txt

# Freeze exact versions
# pip freeze > requirements.txt

# requirements.txt format:
# requests==2.31.0
# boto3>=1.28.0,<2.0.0
# pyyaml
# kubernetes==28.1.0

# ---- IMPORTLIB for dynamic imports ----
import importlib

def load_plugin(plugin_name: str):
    """Dynamically load a plugin module."""
    module = importlib.import_module(f"plugins.{plugin_name}")
    return module.Plugin()

# ---- PACKAGE STRUCTURE ----
# my_sre_tools/
# ├── __init__.py
# ├── deploy/
# │   ├── __init__.py
# │   ├── kubernetes.py
# │   └── canary.py
# ├── monitoring/
# │   ├── __init__.py
# │   └── prometheus.py
# └── utils/
#     ├── __init__.py
#     ├── retry.py
#     └── config.py

# Relative imports within a package
# In my_sre_tools/deploy/kubernetes.py:
# from ..utils.retry import retry
# from ..monitoring.prometheus import push_metric
```

💬 Short Interview Answer

"Python modules are compiled and cached in `sys.modules` on first import. Virtual environments with `venv` are essential for dependency isolation between projects. For SRE scripts, I use requirements.txt with pinned versions for reproducible deployments, and I always import stdlib modules first, then third-party, then local — PEP 8 convention. For packages that are slow to import (boto3 takes ~2 seconds), I sometimes use lazy imports to keep CLI startup times fast."

⚠ Tricky Gotchas

- **Circular imports** — module A imports B which imports A — use local imports inside functions to break the cycle
- `from module import *` **in production** — pollutes namespace and makes code hard to read; never do this in production scripts
- `__init__.py` **can be empty or can re-export** — importing from `package.module` vs just `package` depends on what `__init__.py` exports
- **pip install vs pip3** — on systems with both Python 2 and 3, always use `pip3` or `python3 -m pip`

🔗 DevOps Connection

Packaging your SRE tools as proper Python packages (with `setup.py` or `pyproject.toml`) lets you install them across teams with `pip install git+https://...`. Virtual environments are equivalent to containers for Python dependencies — reproducible, isolated, version-locked.

---

8. Data Structures

📖 What It Is (Simple Terms)

Python's built-in data structures — list, dict, set, tuple — plus `collections` module extensions (deque, Counter, defaultdict, OrderedDict) are the containers you'll use to hold service inventories, metric data, configuration, pod lists, and more.

⚙️ How It Works Internally

- **list**: Dynamic array. Append is O(1) amortized (doubles capacity). Insert at front is O(n).
- **dict**: Hash table. O(1) average get/set/delete. Python 3.7+ preserves insertion order.
- **set**: Hash table (keys only). O(1) membership test. O(n) for iteration.
- **tuple**: Immutable sequence. Slightly faster than list, hashable (can be dict key).
- **deque**: Doubly-linked list. O(1) append/pop from both ends. O(n) random access.

```
from collections import defaultdict, Counter, deque, OrderedDict,
namedtuple
from heapq import heappush, heappop, nlargest, nsmallest
import bisect

# ---- LIST ----

pods = ["web-abc", "api-def", "db-ghi", "web-xyz", "api-uvw"]

# Slicing
first_three = pods[:3]
last_two = pods[-2:]
every_other = pods[::2]
reversed_list = pods[::-1]

# List as stack (LIFO)
stack = []
stack.append("deploy_v1")
stack.append("deploy_v2")
```

```python
latest = stack.pop()  # "deploy_v2"

# Sorting
pods.sort()  # in-place
pods.sort(key=lambda p: p.split("-")[0])  # sort by service name
sorted_pods = sorted(pods, key=len, reverse=True)  # new list

# ---- DICT ----

service_health = {
    "api-gateway": {"healthy": True, "latency_ms": 12, "replicas": 3},
    "auth-service": {"healthy": False, "latency_ms": None, "replicas": 0},
    "db-proxy": {"healthy": True, "latency_ms": 5, "replicas": 2},
}

# Safe access patterns
latency = service_health.get("api-gateway", {}).get("latency_ms", 0)
latency = service_health.get("api-gateway", {}).get("latency_ms") or 0

# .setdefault() — add key only if missing
service_health.setdefault("cache", {"healthy": None, "latency_ms": None})

# dict.update() — merge another dict
overrides = {"api-gateway": {"replicas": 5}}
service_health.update(overrides)

# Merging dicts (Python 3.9+)
defaults = {"timeout": 30, "retries": 3}
overrides = {"timeout": 60}
config = defaults | overrides  # {"timeout": 60, "retries": 3}

# dict comprehension
unhealthy = {k: v for k, v in service_health.items() if not v["healthy"]}

# Nested dict access with error handling
def deep_get(d: dict, *keys, default=None):
    for key in keys:
        if not isinstance(d, dict):
            return default
        d = d.get(key, default)
    return d

region = deep_get(config, "aws", "region", default="us-east-1")

# ---- defaultdict ----

# Group pods by service
from collections import defaultdict

pod_by_service = defaultdict(list)
for pod in pods:
    service = pod.rsplit("-", 1)[0]
    pod_by_service[service].append(pod)
# {"web": ["web-abc", "web-xyz"], "api": ["api-def", "api-uvw"], ...}
```

```
# Count events without KeyError
error_counts = defaultdict(int)
for log_line in log_lines:
    if "ERROR" in log_line:
        service = extract_service(log_line)
        error_counts[service] += 1

# ---- Counter ----

log_levels = ["ERROR", "INFO", "ERROR", "WARN", "ERROR", "INFO", "INFO"]
counts = Counter(log_levels)
# Counter({'INFO': 3, 'ERROR': 3, 'WARN': 1})

print(counts.most_common(2))    # [('INFO', 3), ('ERROR', 3)]
print(counts["DEBUG"])          # 0 — no KeyError for missing keys

# Count pod status
pod_statuses = ["Running", "Pending", "Running", "CrashLoopBackOff",
"Running"]
status_counts = Counter(pod_statuses)

# ---- SET ----

# Unique values, membership tests
production_hosts = {"web-01", "web-02", "api-01", "db-01"}
monitored_hosts = {"web-01", "web-02", "api-01", "cache-01"}

# Set operations
unmonitored = production_hosts - monitored_hosts   # {db-01}
overlap = production_hosts & monitored_hosts       # {web-01, web-02, api-
01}
all_hosts = production_hosts | monitored_hosts     # union
xor = production_hosts ^ monitored_hosts           # symmetric difference

# Fast membership test — O(1) vs O(n) for list
VALID_REGIONS = {"us-east-1", "eu-west-1", "ap-southeast-1"}
if region not in VALID_REGIONS:
    raise ValueError(f"Invalid region: {region}")

# ---- TUPLE ----

# Use for records that shouldn't change
ServiceEndpoint = namedtuple("ServiceEndpoint", ["host", "port",
"protocol"])
ep = ServiceEndpoint("api.internal", 8080, "https")
print(ep.host)   # "api.internal"
print(ep.port)   # 8080

# Tuple as dict key (lists can't be dict keys — not hashable)
latency_cache = {
    ("us-east-1", "api-gateway"): 12.4,
    ("eu-west-1", "api-gateway"): 45.2,
}
```

```python
# ---- deque ----

# Fixed-size circular buffer for recent events
recent_errors = deque(maxlen=100)  # auto-drops oldest when full
recent_errors.appendleft("latest error")  # add to front

# Efficient queue (FIFO) — O(1) popleft vs O(n) for list
task_queue = deque()
task_queue.append("deploy-api")
task_queue.append("deploy-worker")
next_task = task_queue.popleft()  # "deploy-api"

# ---- HEAP (priority queue) ----

import heapq

# Min-heap for SLO violation tracking
violations = []  # (latency, service_name)
heapq.heappush(violations, (500, "api-gateway"))
heapq.heappush(violations, (1200, "auth-service"))
heapq.heappush(violations, (750, "db-proxy"))

worst = heapq.heappop(violations)  # (500, "api-gateway")
top3_worst = heapq.nlargest(3, violations, key=lambda x: x[0])
```

💬 Short Interview Answer

"The right data structure choice has huge performance implications. For SRE scripts: use sets for O(1) membership tests on allow/deny lists — checking if a host is in a whitelist of 10,000 entries is the same speed as checking 10 entries. Use Counter for log level aggregation. Use defaultdict to avoid KeyError when building grouped structures. Use deque with maxlen for sliding window metrics or recent-event buffers. Use heap if you need the top-N or bottom-N items — it's O(N log K) versus O(N log N) for full sort."

⚠ Tricky Gotchas

- `dict.keys()`, `dict.values()`, `dict.items()` **return views** — they reflect changes to the dict and you can't add/remove during iteration
- **Modifying a list while iterating** — undefined behavior; use comprehension or `list(original)` copy
- `tuple` **is immutable but can contain mutable objects** — `([1,2], 3)` is a valid tuple but the list inside it is mutable
- **Set ordering is not guaranteed** — don't rely on iteration order of sets
- `Counter` **returns 0 for missing keys**, `defaultdict(int)` returns 0, but regular `dict` raises `KeyError`

🔗 DevOps Connection

- `Counter` — aggregate log levels, error codes, pod statuses across thousands of pods
- `defaultdict` — build per-service or per-region groupings from flat monitoring data

- `set` — compare lists of running vs expected services to find outages
- `deque(maxlen=N)` — sliding window for rate calculations (requests per minute)
- `heapq` — efficient top-N alerting (find top 10 highest-latency endpoints)

🟡 INTERMEDIATE TOPICS

---

9. os and sys

📖 What It Is (Simple Terms)

`os` provides a portable interface to operating system functionality — files, directories, processes, environment variables, user info. `sys` provides access to Python's runtime — command-line arguments, stdin/stdout/stderr, Python path, exit codes.

⚙️ How It Works Internally

`os` functions are thin wrappers around POSIX syscalls (`stat()`, `open()`, `fork()`, `exec()`, etc.). `sys` exposes CPython interpreter internals. Both are part of the C extension layer of Python.

🧩 Key Patterns

```
import os
import sys
import stat
import pwd
import grp

# ---- ENVIRONMENT VARIABLES ----

# Get (returns None if missing)
region = os.environ.get("AWS_REGION")

# Get with default
region = os.environ.get("AWS_REGION", "us-east-1")

# Require (raise if missing)
def require_env(name: str) -> str:
    value = os.environ.get(name)
    if value is None:
        print(f"ERROR: Required environment variable {name} is not set",
file=sys.stderr)
        sys.exit(1)
    return value

db_password = require_env("DB_PASSWORD")

# os.environ is a dict-like object
os.environ["NEW_VAR"] = "value"   # set
```

```
del os.environ["NEW_VAR"]         # unset (raises KeyError if missing)
os.unsetenv("NEW_VAR")            # unset without KeyError

# Get ALL env vars (useful for debugging)
for key, value in sorted(os.environ.items()):
    if "SECRET" not in key and "PASSWORD" not in key:  # don't log
secrets!
        print(f"{key}={value}")

# ---- PATH OPERATIONS (use pathlib in new code) ----

os.getcwd()                          # current working directory
os.chdir("/tmp")                     # change directory
os.path.exists("/etc/hosts")         # does path exist?
os.path.isfile("/etc/hosts")         # is it a file?
os.path.isdir("/etc")                # is it a directory?
os.path.join("/var", "log", "app")   # join path components
os.path.basename("/var/log/app.log") # "app.log"
os.path.dirname("/var/log/app.log")  # "/var/log"
os.path.abspath("../config.yaml")    # absolute path
os.path.expanduser("~/.kube/config") # expand ~
os.path.expandvars("$HOME/.kube")    # expand env vars

# ---- DIRECTORY OPERATIONS ----

os.listdir("/etc")                    # list directory contents
os.makedirs("/tmp/work/subdir", exist_ok=True)  # mkdir -p
os.makedirs("/tmp/work", mode=0o755, exist_ok=True)
os.rmdir("/tmp/empty_dir")            # remove empty dir
os.unlink("/tmp/file")                # delete file
os.rename("/tmp/old", "/tmp/new")     # rename (atomic on same FS)

# Walk directory tree
for root, dirs, files in os.walk("/app"):
    # Prune directories (modify dirs in-place to skip them)
    dirs[:] = [d for d in dirs if d != ".git"]
    for file in files:
        full_path = os.path.join(root, file)
        if file.endswith(".log"):
            print(full_path)

# ---- FILE METADATA ----

st = os.stat("/etc/hosts")
print(f"Size: {st.st_size} bytes")
print(f"Modified: {st.st_mtime}")
print(f"Mode: {oct(st.st_mode)}")
print(f"Owner UID: {st.st_uid}")

# Check permissions
is_readable = os.access("/etc/secret", os.R_OK)
is_writable = os.access("/etc/secret", os.W_OK)
is_executable = os.access("/usr/bin/kubectl", os.X_OK)
```

```python
# Check file mode bits
mode = os.stat("/etc/passwd").st_mode
is_world_readable = bool(mode & stat.S_IROTH)
if is_world_readable:
    print("WARNING: /etc/passwd is world-readable")

# ---- PROCESS OPERATIONS ----

os.getpid()     # current process ID
os.getppid()    # parent process ID
os.getuid()     # current user ID
os.getgid()     # current group ID
os.getlogin()   # login name

# User/group info
user_info = pwd.getpwuid(os.getuid())
print(f"Running as: {user_info.pw_name} (UID {user_info.pw_uid})")

# ---- sys MODULE ----

sys.argv         # command-line arguments ['script.py', 'arg1', 'arg2']
sys.stdin        # standard input
sys.stdout       # standard output
sys.stderr       # standard error (always write errors here!)
sys.exit(0)      # exit with code (0=success, non-zero=error)
sys.version      # Python version string
sys.platform     # 'linux', 'darwin', 'win32'
sys.executable   # path to python interpreter
sys.prefix       # Python installation prefix

# Write to stderr (doesn't go to stdout pipeline)
print("Error: config not found", file=sys.stderr)

# Flush output (important for real-time log streaming in containers)
print("Deploying...", flush=True)
sys.stdout.flush()

# Redirect stdout/stderr
import io
buffer = io.StringIO()
old_stdout = sys.stdout
sys.stdout = buffer
print("captured")
sys.stdout = old_stdout
output = buffer.getvalue()  # "captured\n"

# sys.path manipulation (add to import path)
sys.path.insert(0, "/app/lib")

# ---- SIGNALS ----
import signal

def graceful_shutdown(signum, frame):
    print("\nGraceful shutdown initiated...", file=sys.stderr)
```

```
    cleanup()
    sys.exit(0)

signal.signal(signal.SIGTERM, graceful_shutdown)  # kill command
signal.signal(signal.SIGINT, graceful_shutdown)   # Ctrl+C

# Set timeout using SIGALRM (Unix only)
def timeout_handler(signum, frame):
    raise TimeoutError("Operation timed out")

signal.signal(signal.SIGALRM, timeout_handler)
signal.alarm(30)  # raise SIGALRM in 30 seconds
try:
    result = long_running_operation()
finally:
    signal.alarm(0)  # cancel alarm
```

💬 Short Interview Answer

"For SRE scripts, `os` and `sys` are foundational. Key patterns: always use `os.environ.get()` with a default rather than `os.environ[]` which raises KeyError; write errors to `sys.stderr` so they don't pollute stdout in pipeline scripts; use `sys.exit()` with meaningful exit codes for CI/CD — 0 for success, anything else for failure; handle `SIGTERM` gracefully in long-running scripts because that's what Kubernetes sends before SIGKILL. For container environments, always flush stdout (`print(..., flush=True)`) because Python buffers output by default."

⚠ Tricky Gotchas

- **`os.environ` changes are inherited by child processes** (via `subprocess`) but NOT by the parent process
- **`sys.exit()` raises `SystemExit`** — it can be caught by `except Exception`? NO — `SystemExit` inherits from `BaseException`. But a bare `except:` WILL catch it.
- **Python buffers stdout in non-TTY mode** (e.g., when piped to `tee`) — always use `flush=True` or `PYTHONUNBUFFERED=1` in containers
- **`os.path.join("/a/b", "/c/d")` returns `/c/d`** — an absolute second argument discards everything before it

🔗 DevOps Connection

- SIGTERM handling is critical in Kubernetes — pods receive SIGTERM before being killed; your script must flush logs, close DB connections, deregister from service discovery
- Exit codes are how CI/CD pipelines (Jenkins, GitLab, GitHub Actions) know if a step passed or failed
- Environment variables are how 12-factor apps receive configuration in containers

---

10. subprocess

📖 What It Is (Simple Terms)

`subprocess` lets Python scripts run external commands — `kubectl`, `helm`, `terraform`, `git`, `aws`, `docker`, `ansible` — and capture their output, check their exit codes, and feed them input. It's your bridge between Python automation and existing CLI tools.

⚙️ How It Works Internally

`subprocess.run()` forks the current process, `exec()`s the command in the child, and waits for it to finish. `Popen` is the lower-level class that gives you fine-grained control over stdin/stdout/stderr pipes. Pipes have a fixed buffer size (~64KB) — if you don't read from stdout, the child process may block waiting for the pipe to drain.

🧩 Key Patterns

```python
import subprocess
import shlex
import os
from typing import Optional

# ---- BASIC USAGE ----

# subprocess.run() — recommended for most cases
result = subprocess.run(
    ["kubectl", "get", "pods", "-n", "production"],
    capture_output=True,   # capture stdout and stderr
    text=True,             # decode as string (vs bytes)
    check=True,            # raise CalledProcessError if exit code != 0
    timeout=30,            # raise TimeoutExpired after 30 seconds
)
print(result.stdout)
print(result.returncode)  # 0

# ---- check=False for handling non-zero exit codes ----
result = subprocess.run(
    ["kubectl", "get", "deployment", "my-app", "-n", "prod"],
    capture_output=True, text=True, check=False
)
if result.returncode == 0:
    print("Deployment exists")
elif result.returncode == 1 and "not found" in result.stderr:
    print("Deployment does not exist yet")
else:
    raise RuntimeError(f"kubectl failed: {result.stderr}")

# ---- Error handling ----
try:
    result = subprocess.run(
        ["helm", "upgrade", "--install", "my-app", "./chart"],
        capture_output=True, text=True, check=True, timeout=120
    )
except subprocess.CalledProcessError as e:
    print(f"Command failed with exit code {e.returncode}")
    print(f"stdout: {e.stdout}")
```

```
    print(f"stderr: {e.stderr}")
    raise
except subprocess.TimeoutExpired as e:
    print(f"Command timed out after {e.timeout}s")
    raise

# ---- NEVER DO THIS (shell injection vulnerability) ----
# ⚠ DANGEROUS — user input could be "web-01; rm -rf /"
host = user_input()
os.system(f"ping -c 1 {host}")       # BAD
subprocess.run(f"ping -c 1 {host}", shell=True)  # BAD

# ✅ SAFE — always use list form
subprocess.run(["ping", "-c", "1", host])

# ⚠ If you MUST use shell=True (e.g., for pipe chains):
# Use shlex.quote() to escape user input
cmd = f"grep {shlex.quote(search_term)} /var/log/app.log | tail -100"
subprocess.run(cmd, shell=True)

# ---- STREAMING OUTPUT (for long-running commands) ----
import sys

def run_streaming(cmd: list[str]) -> int:
    """Run command with real-time output streaming."""
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,  # merge stderr into stdout
        text=True,
        bufsize=1,  # line-buffered
    )

    for line in process.stdout:
        print(line, end="", flush=True)

    process.wait()
    return process.returncode

exit_code = run_streaming(["kubectl", "rollout", "status",
"deployment/api", "--watch"])

# ---- FEEDING INPUT TO STDIN ----
manifest = """
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  key: value
"""
result = subprocess.run(
    ["kubectl", "apply", "-f", "-"],
    input=manifest,
```

```python
    capture_output=True,
    text=True,
    check=True,
)

# ---- ENVIRONMENT VARIABLES FOR SUBPROCESS ----
env = os.environ.copy()  # inherit current env
env["KUBECONFIG"] = "/tmp/prod-kubeconfig"
env["AWS_PROFILE"] = "production"

result = subprocess.run(
    ["kubectl", "get", "nodes"],
    env=env,
    capture_output=True, text=True, check=True
)

# ---- PIPELINE (equivalent to cmd1 | cmd2) ----
proc1 = subprocess.Popen(
    ["kubectl", "logs", "pod/my-pod", "--tail=1000"],
    stdout=subprocess.PIPE, text=True
)
proc2 = subprocess.Popen(
    ["grep", "ERROR"],
    stdin=proc1.stdout,
    stdout=subprocess.PIPE, text=True
)
proc1.stdout.close()  # allow proc1 to receive SIGPIPE if proc2 exits
output, _ = proc2.communicate()
print(output)

# ---- ASYNC SUBPROCESS ----
import asyncio

async def run_async(cmd: list[str]) -> tuple[int, str, str]:
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    stdout, stderr = await proc.communicate()
    return proc.returncode, stdout.decode(), stderr.decode()

async def check_all_services(services: list[str]) -> dict:
    """Check multiple services concurrently."""
    tasks = {
        svc: asyncio.create_task(
            run_async(["curl", "-sf", f"http://{svc}/health"])
        )
        for svc in services
    }
    results = {}
    for svc, task in tasks.items():
        code, stdout, stderr = await task
        results[svc] = code == 0
```

```
    return results

# ---- UTILITY: RUN WITH RETRIES ----
def run_with_retry(
    cmd: list[str],
    max_attempts: int = 3,
    delay: float = 5.0,
    **kwargs
) -> subprocess.CompletedProcess:
    for attempt in range(1, max_attempts + 1):
        try:
            return subprocess.run(cmd, **kwargs)
        except (subprocess.CalledProcessError, subprocess.TimeoutExpired)
as e:
            if attempt == max_attempts:
                raise
            print(f"Attempt {attempt} failed: {e}. Retrying in
{delay}s...")
            import time; time.sleep(delay)
```

💬 Short Interview Answer

"The cardinal rule with subprocess is: always pass commands as a list, never as a string with `shell=True` unless you've explicitly quoted user input with `shlex.quote()`. Use `capture_output=True, text=True, check=True` for most commands. For long-running commands like `kubectl rollout status --watch`, use `Popen` and stream line by line to avoid buffering the entire output. Always set a timeout — if kubectl hangs, your deployment script shouldn't hang indefinitely. The `input=` parameter lets you pipe data to stdin, which is how you do `kubectl apply -f -` from Python."

⚠ Tricky Gotchas

- **Shell injection** with `shell=True` and unquoted user input — always use list form
- **Pipe buffer deadlock**: if you use `Popen` with `PIPE` for both stdout and stderr and the child writes more than ~64KB to either, it may block waiting for you to read — use `communicate()` or thread readers
- **`check=True` raises on exit code != 0** — but kubectl returns 1 for "resource not found" which is not always an error
- **`subprocess.run()` is synchronous** — it blocks the calling thread until the command finishes
- Environment variables set in a subprocess DO NOT propagate back to the parent Python process

🔗 DevOps Connection

Every SRE automation script that wraps `kubectl`, `helm`, `terraform`, `aws`, `gcloud`, `git`, or `ansible` uses subprocess. The streaming pattern is essential for kubectl rollout watching in CI/CD pipelines. The input= pattern enables `kubectl apply` from dynamically generated manifests.

---

11. pathlib

📖 What It Is (Simple Terms)

`pathlib` provides an object-oriented interface to filesystem paths. Instead of string concatenation with `os.path.join()`, you use `/` operator on `Path` objects. Introduced in Python 3.4, it's the modern way to handle paths.

⚙️ How It Works Internally

`Path` is an abstract base class. `PurePosixPath` and `PureWindowsPath` do pure path manipulation without I/O. `PosixPath` and `WindowsPath` add I/O operations. On instantiation, `Path()` returns the correct platform-specific subclass.

```
from pathlib import Path
import os

# ---- BASIC USAGE ----

# Create paths
home = Path.home()                    # /home/ubuntu
cwd = Path.cwd()                      # current directory
config = Path("/etc/myapp/config.yaml")
relative = Path("configs/prod.yaml")
absolute = relative.resolve()         # converts to absolute

# Path composition with / operator
kube_config = Path.home() / ".kube" / "config"
log_dir = Path("/var/log") / "myapp"

# String representation
print(str(config))       # "/etc/myapp/config.yaml"
print(config.as_posix()) # "/etc/myapp/config.yaml"

# ---- PATH COMPONENTS ----

p = Path("/var/log/myapp/error.log.2024")

p.name         # "error.log.2024"        (filename with all extensions)
p.stem         # "error.log"             (filename without LAST suffix)
p.suffix       # ".2024"                 (last extension)
p.suffixes     # ['.log', '.2024']        (all extensions)
p.parent       # Path('/var/log/myapp')
p.parents[0]   # Path('/var/log/myapp')
p.parents[1]   # Path('/var/log')
p.parts        # ('/', 'var', 'log', 'myapp', 'error.log.2024')
p.root         # '/'
p.anchor       # '/'

# ---- FILE SYSTEM OPERATIONS ----

config_path = Path("/etc/myapp/config.yaml")

# Existence and type checks
config_path.exists()     # True/False
config_path.is_file()    # True if regular file
```

```
config_path.is_dir()     # True if directory
config_path.is_symlink() # True if symlink

# Create directories
log_dir = Path("/var/log/myapp/2024")
log_dir.mkdir(parents=True, exist_ok=True)  # mkdir -p

# File operations
config_path.read_text(encoding="utf-8")     # read entire file as string
config_path.read_bytes()                    # read as bytes
config_path.write_text("content", encoding="utf-8")  # write string
config_path.write_bytes(b"binary content") # write bytes

# Rename/move
config_path.rename(config_path.with_suffix(".yaml.bak"))
config_path.replace(Path("/new/location/config.yaml"))  # replace existing

# Delete
config_path.unlink()                        # delete file
config_path.unlink(missing_ok=True)         # delete, don't raise if
missing
log_dir.rmdir()                             # delete EMPTY directory

# Stat
stat = config_path.stat()
size = stat.st_size
mtime = stat.st_mtime

# ---- GLOB AND RGLOB ----

app_dir = Path("/app")

# Find all Python files
py_files = list(app_dir.glob("*.py"))         # non-recursive
all_py_files = list(app_dir.rglob("*.py"))    # recursive

# Find all log files older than 7 days
import time
cutoff = time.time() - 7 * 24 * 3600
old_logs = [
    f for f in Path("/var/log").rglob("*.log")
    if f.stat().st_mtime < cutoff
]

# ---- PATH MANIPULATION ----

p = Path("/var/log/app.log")

p.with_name("error.log")        # /var/log/error.log
p.with_suffix(".json")          # /var/log/app.json
p.with_stem("debug")            # /var/log/debug.log (Python 3.9+)

# Relative paths
base = Path("/var/log")
```

```python
full = Path("/var/log/app/error.log")
relative = full.relative_to(base)  # app/error.log

# ---- PRACTICAL SRE PATTERNS ----

def find_kubeconfig(contexts: list[str]) -> dict[str, Path]:
    """Find kubeconfig files for given context names."""
    kube_dir = Path.home() / ".kube"
    configs = {}
    for ctx in contexts:
        # Check multiple conventions
        for pattern in [f"{ctx}.yaml", f"{ctx}", f"config-{ctx}"]:
            candidate = kube_dir / pattern
            if candidate.exists():
                configs[ctx] = candidate
                break
    return configs

def rotate_log(log_path: Path, max_backups: int = 5) -> None:
    """Rotate a log file, keeping max_backups copies."""
    if not log_path.exists():
        return

    # Shift existing backups
    for i in range(max_backups - 1, 0, -1):
        old = log_path.with_suffix(f".{i}")
        new = log_path.with_suffix(f".{i+1}")
        if old.exists():
            old.rename(new)

    # Rename current log to .1
    log_path.rename(log_path.with_suffix(".1"))

    # Create fresh log file
    log_path.touch()

def cleanup_temp_dirs(base_dir: Path, pattern: str = "deploy-*") -> int:
    """Remove old temporary deployment directories."""
    removed = 0
    for d in base_dir.glob(pattern):
        if d.is_dir():
            import shutil
            shutil.rmtree(d)
            removed += 1
    return removed
```

💬 Short Interview Answer

"pathlib is the modern Pythonic way to work with filesystem paths. The key advantage is composability — you can use the `/` operator to build paths, call methods directly on the path object like `path.read_text()` instead of `open(path).read()`, and use `rglob('*.yaml')` to find files

recursively. `Path.mkdir(parents=True, exist_ok=True)` is the clean equivalent of `mkdir -p`. I prefer pathlib over `os.path` in all new code — it's more readable and object-oriented."

⚠ Tricky Gotchas

- `Path("/a") / "/b"` returns `Path("/b")` — absolute path on right side discards left side (same as `os.path.join`)
- `path.stem` removes only the LAST suffix — `Path("app.log.2024").stem` is `"app.log"`, not `"app"`
- `path.glob("**/*.yaml")` requires the `**` component — plain `*.yaml` is not recursive
- `path.unlink()` raises `FileNotFoundError` by default in Python < 3.8; use `missing_ok=True` in 3.8+

🔗 DevOps Connection

Config file management, log rotation, deployment artifact handling, kubeconfig management, Ansible inventory file discovery — all cleaner with pathlib than raw string operations.

---

12. logging

📖 What It Is (Simple Terms)

Python's `logging` module is the production-grade way to emit log messages from scripts and services. Unlike `print()`, it supports log levels, multiple output destinations (handlers), structured formatting, log rotation, and can be configured at runtime without code changes.

⚙️ How It Works Internally

The logging system has a hierarchy of **loggers** (tree rooted at the root logger), **handlers** (where to write: file, stderr, syslog, HTTP), and **formatters** (how to format). A logger passes records up to parent loggers unless `propagate=False`. The effective level is the first non-NOTSET level found walking up the tree.

```
import logging
import logging.handlers
import sys
import json
from datetime import datetime

# ---- BASIC SETUP ----

# Simplest setup (don't use in production)
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)  # use module name as logger name

# ---- PRODUCTION SETUP ----

def setup_logging(
    level: str = "INFO",
    json_format: bool = False,
```

```
    log_file: str | None = None
) -> logging.Logger:
    """Configure production-ready logging."""

    logger = logging.getLogger()  # root logger
    logger.setLevel(getattr(logging, level.upper()))

    # Remove existing handlers
    logger.handlers.clear()

    if json_format:
        formatter = JsonFormatter()
    else:
        formatter = logging.Formatter(
            fmt="%(asctime)s %(levelname)-8s %(name)s %(message)s",
            datefmt="%Y-%m-%dT%H:%M:%S"
        )

    # Console handler (stderr)
    console = logging.StreamHandler(sys.stderr)
    console.setFormatter(formatter)
    logger.addHandler(console)

    # File handler with rotation
    if log_file:
        file_handler = logging.handlers.RotatingFileHandler(
            log_file,
            maxBytes=100 * 1024 * 1024,  # 100MB
            backupCount=5,
        )
        file_handler.setFormatter(formatter)
        logger.addHandler(file_handler)

    return logger

# ---- JSON STRUCTURED LOGGING ----

class JsonFormatter(logging.Formatter):
    """Emit logs as JSON for log aggregators (Datadog, ELK,
CloudWatch)."""

    def format(self, record: logging.LogRecord) -> str:
        log_data = {
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
            "line": record.lineno,
        }

        # Include exception info if present
        if record.exc_info:
```

```
            log_data["exception"] = self.formatException(record.exc_info)

        # Include extra fields
        for key, value in record.__dict__.items():
            if key not in logging.LogRecord.__dict__ and not
key.startswith("_"):
                log_data[key] = value

        return json.dumps(log_data)

# ---- USAGE PATTERNS ----

logger = logging.getLogger(__name__)

# Basic levels
logger.debug("Debug details: pod=%s phase=%s", pod_name, phase)
logger.info("Deployment started: %s → %s", old_version, new_version)
logger.warning("High latency detected: %.1fms (threshold: %dms)", latency,
threshold)
logger.error("Health check failed: %s returned %d", url, status_code)
logger.critical("DATABASE CONNECTION LOST — initiating failover")

# Log with exception (includes full traceback)
try:
    result = call_api()
except Exception:
    logger.exception("API call failed")  # automatically captures exc_info

# Log with extra structured fields (useful for JSON logging)
logger.info(
    "Deployment complete",
    extra={
        "service": "api-gateway",
        "version": "v1.2.3",
        "duration_seconds": 45.2,
        "replicas": 3,
        "environment": "production",
    }
)

# ---- LOG LEVELS ----
# DEBUG    10  — verbose debugging info
# INFO     20  — normal operation events
# WARNING  30  — something unexpected but non-fatal
# ERROR    40  — serious failure, operation aborted
# CRITICAL 50  — system-level failure, may need immediate attention

# ---- CONTEXT LOGGING ----

import contextvars
import logging

# Use contextvars to add request/trace ID to all logs in a context
request_id_var = contextvars.ContextVar("request_id", default="-")
```

```python
class ContextFilter(logging.Filter):
    def filter(self, record):
        record.request_id = request_id_var.get()
        return True

def configure_with_context():
    handler = logging.StreamHandler()
    handler.setFormatter(
        logging.Formatter("%(asctime)s [%(request_id)s] %(levelname)s %
(message)s")
    )
    handler.addFilter(ContextFilter())
    logging.getLogger().addHandler(handler)

# ---- SUPPRESS NOISY LIBRARIES ----

# Suppress boto3/urllib3/requests noise in production logs
logging.getLogger("boto3").setLevel(logging.WARNING)
logging.getLogger("botocore").setLevel(logging.WARNING)
logging.getLogger("urllib3").setLevel(logging.WARNING)
logging.getLogger("kubernetes").setLevel(logging.WARNING)
logging.getLogger("paramiko").setLevel(logging.WARNING)

# ---- SYSLOG HANDLER ----

syslog_handler = logging.handlers.SysLogHandler(address='/dev/log')
syslog_handler.setFormatter(logging.Formatter('%(name)s: %(message)s'))

# ---- TIMED ROTATING FILE HANDLER ----
handler = logging.handlers.TimedRotatingFileHandler(
    "/var/log/myapp/app.log",
    when="midnight",     # rotate daily at midnight
    interval=1,
    backupCount=30,      # keep 30 days of logs
    utc=True,
)
```

💬 Short Interview Answer

"Production Python scripts should use the `logging` module, never `print()`. Key setup: get a logger with `logging.getLogger(__name__)` in each module, configure the root logger once at startup with `basicConfig` or a custom setup. For containers, use a `StreamHandler` to stderr with JSON format so log aggregators like Datadog or ELK can parse structured fields. Always suppress verbose third-party loggers like `boto3` and `urllib3` which would otherwise flood your logs. Use `logger.exception()` inside except blocks — it automatically captures the full traceback."

⚠ Tricky Gotchas

- **`logging.basicConfig()` only works if root logger has no handlers** — if boto3 imported before you called basicConfig, it may have already added a handler

- **Don't use `%` string formatting in log calls** — use `logger.info("msg %s", var)` not `logger.info(f"msg {var}")` — the lazy formatting only happens if the message will actually be emitted
- **`propagate=True` by default** — if you add handlers to both root and a child logger, messages may appear twice
- **Log levels are inherited**: a child logger with level NOTSET inherits from parent — setting level on root logger affects all loggers

🔗 DevOps Connection

Structured JSON logs enable correlation in ELK/Datadog/Splunk. Log rotation prevents disk full incidents. Suppressing boto3 noise keeps CloudWatch costs down. Context variables enable distributed tracing — adding a trace ID to every log line during a request.

---

13. argparse

📖 What It Is (Simple Terms)

`argparse` is Python's standard library for building command-line interfaces. It parses `sys.argv`, validates arguments, generates help text, and handles subcommands. It's how you turn a Python script into a real CLI tool.

⚙️ How It Works Internally

`argparse` builds an argument namespace by processing `sys.argv[1:]` against registered argument patterns. It handles type conversion, required arguments, mutually exclusive groups, and subparsers (for multi-command CLIs like `git commit`, `git push`).

```
import argparse
import sys
import os

# ---- BASIC PARSER ----

def create_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="deploy",
        description="Deploy services to Kubernetes",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Examples:
  deploy --service api-gateway --version v1.2.3
  deploy --service api-gateway --version v1.2.3 --dry-run
  deploy --service api-gateway --version v1.2.3 -n production
        """
    )

    # Required positional argument
    parser.add_argument(
```

```
        "service",
        help="Service name to deploy",
    )

    # Optional with default
    parser.add_argument(
        "--version", "-v",
        required=True,
        help="Image version/tag to deploy (e.g., v1.2.3)"
    )

    parser.add_argument(
        "--namespace", "-n",
        default="default",
        help="Kubernetes namespace (default: %(default)s)"
    )

    # Boolean flag
    parser.add_argument(
        "--dry-run",
        action="store_true",
        default=False,
        help="Print what would be done without executing"
    )

    # Integer with range validation
    parser.add_argument(
        "--replicas",
        type=int,
        default=None,
        metavar="N",
        help="Number of replicas (default: keep current)"
    )

    # Choices
    parser.add_argument(
        "--environment",
        choices=["staging", "production", "canary"],
        default="staging",
        help="Target environment"
    )

    # Multiple values
    parser.add_argument(
        "--labels",
        nargs="*",
        metavar="KEY=VALUE",
        help="Additional labels (e.g., --labels team=platform version=v1)"
    )

    # Append multiple times
    parser.add_argument(
        "--extra-arg",
        action="append",
```

```
        dest="extra_args",
        help="Extra helm args (can be specified multiple times)"
    )

    # Log level
    parser.add_argument(
        "--log-level",
        choices=["DEBUG", "INFO", "WARNING", "ERROR"],
        default="INFO",
        help="Log verbosity (default: %(default)s)"
    )

    return parser

# ---- SUBCOMMANDS ----

def create_full_cli() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="sretool", description="SRE
Toolbox")
    subparsers = parser.add_subparsers(dest="command", metavar="COMMAND")
    subparsers.required = True  # require a subcommand

    # deploy subcommand
    deploy_parser = subparsers.add_parser("deploy", help="Deploy a
service")
    deploy_parser.add_argument("service", help="Service name")
    deploy_parser.add_argument("--version", required=True)
    deploy_parser.set_defaults(func=handle_deploy)

    # rollback subcommand
    rollback_parser = subparsers.add_parser("rollback", help="Roll back a
deployment")
    rollback_parser.add_argument("service")
    rollback_parser.add_argument("--revision", type=int, help="Target
revision number")
    rollback_parser.set_defaults(func=handle_rollback)

    # status subcommand
    status_parser = subparsers.add_parser("status", help="Check service
status")
    status_parser.add_argument("services", nargs="+", help="Service names
to check")
    status_parser.add_argument("--watch", action="store_true")
    status_parser.set_defaults(func=handle_status)

    return parser

def handle_deploy(args):
    print(f"Deploying {args.service} @ {args.version}")

def handle_rollback(args):
    print(f"Rolling back {args.service} to revision {args.revision}")

def handle_status(args):
```

```
    print(f"Checking status of: {', '.join(args.services)}")

# ---- CUSTOM VALIDATION ----

def parse_label(s: str) -> tuple[str, str]:
    """Parse 'key=value' label string."""
    parts = s.split("=", 1)
    if len(parts) != 2:
        raise argparse.ArgumentTypeError(
            f"Invalid label format: '{s}'. Expected KEY=VALUE"
        )
    return tuple(parts)

def positive_int(value: str) -> int:
    """Validate that value is a positive integer."""
    try:
        ivalue = int(value)
    except ValueError:
        raise argparse.ArgumentTypeError(f"'{value}' is not a valid
integer")
    if ivalue <= 0:
        raise argparse.ArgumentTypeError(f"Replicas must be positive, got
{ivalue}")
    return ivalue

# Use custom type
parser.add_argument("--replicas", type=positive_int)

# ---- MUTUAL EXCLUSION ----

group = parser.add_mutually_exclusive_group()
group.add_argument("--verbose", "-v", action="store_true")
group.add_argument("--quiet", "-q", action="store_true")

# ---- MAIN ENTRY POINT PATTERN ----

def main():
    parser = create_parser()
    args = parser.parse_args()

    # Setup logging based on args
    import logging
    logging.basicConfig(level=args.log_level)

    # Parse labels if provided
    labels = {}
    if args.labels:
        for label in args.labels:
            k, v = label.split("=", 1)
            labels[k] = v

    if args.dry_run:
        print(f"[DRY RUN] Would deploy {args.service}:{args.version} "
              f"to {args.namespace}")
```

```
        return 0

    # do actual deployment
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

💬 Short Interview Answer

"argparse is the standard way to build CLI tools in Python. Key patterns: always use `--long-form` flags with short `-l` aliases, use `action='store_true'` for boolean flags, use `choices=[]` for enum-like args (environment, log level), use `type=int` or custom type functions for validation, and use subparsers for multi-command tools like `deploy`, `rollback`, `status`. The `%(default)s` in help text automatically inserts the default value. Always call `sys.exit(main())` so the process exits with the right code."

⚠ Tricky Gotchas

- `nargs="+"` requires at least one value; `nargs="*"` allows zero; `nargs="?"` allows zero or one
- `action="append"` starts as `None` if not used, not `[]` — check `if args.extra_args`
- `subparsers.required = True` needed if you want a subcommand to be mandatory (not required by default in Python 3.3+)
- `dest` on `add_argument` controls the attribute name in the `args` namespace — `--dry-run` becomes `args.dry_run` (hyphen → underscore) automatically

🔗 DevOps Connection

argparse is the foundation of every SRE CLI tool — deployment scripts, runbook automation, inventory scripts, incident response tools. Tools built with argparse are self-documenting via `--help` and can be called from CI/CD pipelines with deterministic arguments.

---

14. Environment Variables and Config Management

📖 What It Is (Simple Terms)

Config management in Python means reading configuration from environment variables, files (YAML, JSON, TOML, INI), and secret stores in a clean, validated, and maintainable way. The 12-factor app pattern mandates storing config in the environment — Python needs clean patterns to implement this.

🧩 Key Patterns

```
import os
import json
import configparser
from dataclasses import dataclass, field
from typing import Optional
from pathlib import Path
```

```python
# ---- SIMPLE ENV VAR LOADING ----

class ConfigError(Exception):
    pass

def get_env(name: str, default=None, required: bool = False) -> str |
None:
    value = os.environ.get(name, default)
    if required and value is None:
        raise ConfigError(f"Required environment variable '{name}' not
set")
    return value

# ---- TYPED CONFIG WITH DATACLASS ----

@dataclass
class DatabaseConfig:
    host: str
    port: int
    name: str
    user: str
    password: str
    pool_size: int = 10
    connect_timeout: int = 30
    ssl_mode: str = "require"

    @classmethod
    def from_env(cls) -> "DatabaseConfig":
        return cls(
            host=get_env("DB_HOST", required=True),
            port=int(get_env("DB_PORT", "5432")),
            name=get_env("DB_NAME", required=True),
            user=get_env("DB_USER", required=True),
            password=get_env("DB_PASSWORD", required=True),
            pool_size=int(get_env("DB_POOL_SIZE", "10")),
            connect_timeout=int(get_env("DB_CONNECT_TIMEOUT", "30")),
            ssl_mode=get_env("DB_SSL_MODE", "require"),
        )

    @property
    def dsn(self) -> str:
        return (f"postgresql://{self.user}:{self.password}@"
                f"{self.host}:{self.port}/{self.name}"
                f"?sslmode={self.ssl_mode}")

@dataclass
class AppConfig:
    environment: str
    region: str
    log_level: str = "INFO"
    database: DatabaseConfig =
field(default_factory=DatabaseConfig.from_env)
    feature_flags: dict = field(default_factory=dict)
```

```python
    @classmethod
    def from_env(cls) -> "AppConfig":
        env = get_env("ENVIRONMENT", required=True)
        if env not in ("development", "staging", "production"):
            raise ConfigError(f"Invalid ENVIRONMENT: {env}")

        # Parse feature flags from JSON env var
        flags_json = get_env("FEATURE_FLAGS", "{}")
        try:
            flags = json.loads(flags_json)
        except json.JSONDecodeError as e:
            raise ConfigError(f"Invalid FEATURE_FLAGS JSON: {e}")

        return cls(
            environment=env,
            region=get_env("AWS_REGION", "us-east-1"),
            log_level=get_env("LOG_LEVEL", "INFO"),
            feature_flags=flags,
        )

# ---- YAML CONFIG FILES ----

import yaml

def load_yaml_config(path: str, env_override: bool = True) -> dict:
    """Load YAML config with optional env var overrides."""
    config_path = Path(path)
    if not config_path.exists():
        raise ConfigError(f"Config file not found: {path}")

    with open(config_path) as f:
        config = yaml.safe_load(f) or {}

    if env_override:
        # Allow env vars to override config file values
        # e.g., MYAPP_DATABASE__HOST overrides config.database.host
        prefix = "MYAPP_"
        for key, value in os.environ.items():
            if key.startswith(prefix):
                config_key = key[len(prefix):].lower().replace("__", ".")
                set_nested(config, config_key.split("."), value)

    return config

def set_nested(d: dict, keys: list[str], value) -> None:
    """Set a nested dict value given a list of keys."""
    for key in keys[:-1]:
        d = d.setdefault(key, {})
    d[keys[-1]] = value

# ---- .env FILE LOADING ----

def load_dotenv(env_file: str = ".env") -> None:
    """Load a .env file into os.environ (minimal version of python-
```

```
dotenv)."""
    path = Path(env_file)
    if not path.exists():
        return

    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            if "=" not in line:
                continue
            key, value = line.split("=", 1)
            key = key.strip()
            value = value.strip().strip('"').strip("'")
            os.environ.setdefault(key, value)  # don't override existing
env vars

# ---- CONFIGPARSER (INI files) ----

config = configparser.ConfigParser()
config.read("/etc/myapp/config.ini")

db_host = config.get("database", "host", fallback="localhost")
db_port = config.getint("database", "port", fallback=5432)
debug_mode = config.getboolean("app", "debug", fallback=False)

# ---- PYDANTIC SETTINGS (recommended for production) ----

# pip install pydantic-settings
from pydantic_settings import BaseSettings
from pydantic import Field, validator

class Settings(BaseSettings):
    environment: str = Field(..., env="ENVIRONMENT")
    aws_region: str = Field("us-east-1", env="AWS_REGION")
    db_host: str = Field(..., env="DB_HOST")
    db_port: int = Field(5432, env="DB_PORT")
    db_password: str = Field(..., env="DB_PASSWORD")
    log_level: str = Field("INFO", env="LOG_LEVEL")

    @validator("environment")
    def validate_environment(cls, v):
        if v not in ("development", "staging", "production"):
            raise ValueError(f"Invalid environment: {v}")
        return v

    @validator("log_level")
    def validate_log_level(cls, v):
        return v.upper()

    class Config:
        env_file = ".env"       # also load from .env file
        case_sensitive = False  # ENV_VAR and env_var both work
```

```
# Singleton pattern for config
_settings: Settings | None = None

def get_settings() -> Settings:
    global _settings
    if _settings is None:
        _settings = Settings()
    return _settings
```

💬 Short Interview Answer

"Config management follows the 12-factor principle: store config in environment variables, not code. For simple scripts I use `os.environ.get()` with defaults and a `require_env()` helper that exits cleanly. For production services, I use Pydantic Settings which validates types, reads from `.env` files, and provides clear error messages when required variables are missing. Key concerns: never log config values that might contain secrets, use `os.environ.setdefault()` (not `os.environ[]=`) so you don't override real env vars with .env file defaults."

⚠️ Tricky Gotchas

- **`os.environ.get()` always returns strings** — must cast `int(os.environ.get("PORT", "8080"))` explicitly
- **Boolean env vars** — `"false"` is truthy in Python! Use `os.environ.get("DEBUG", "false").lower() == "true"`
- **The `.env` file should never be committed** — add to `.gitignore`; secrets in env vars, not files

🔗 DevOps Connection

12-factor apps, Kubernetes ConfigMaps and Secrets, Helm values, Terraform outputs — all feed configuration via environment variables. Pydantic Settings is the clean Pythonic way to consume this in a validated, type-safe way.

---

15. JSON and YAML Parsing

📖 What It Is (Simple Terms)

JSON is the lingua franca of APIs; YAML is the language of Kubernetes, Ansible, Helm, and CI/CD configs. Python's `json` module is in stdlib; `pyyaml` is the standard third-party library for YAML.

🧩 Key Patterns

```
import json
import yaml
from datetime import datetime
from pathlib import Path
from typing import Any
```

```python
# ---- JSON ----

# Parse JSON string
data = json.loads('{"service": "api", "replicas": 3}')

# Parse JSON file
with open("config.json") as f:
    config = json.load(f)

# Serialize to string
json_str = json.dumps(data, indent=2, sort_keys=True)

# Serialize to file
with open("output.json", "w") as f:
    json.dump(data, f, indent=2, sort_keys=True)

# ---- CUSTOM JSON SERIALIZATION ----

class SREJSONEncoder(json.JSONEncoder):
    """Handle types that json module can't serialize by default."""
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        if isinstance(obj, Path):
            return str(obj)
        if isinstance(obj, set):
            return sorted(list(obj))
        if hasattr(obj, '__dict__'):
            return obj.__dict__
        return super().default(obj)

alert = {
    "service": "api-gateway",
    "timestamp": datetime.utcnow(),
    "affected_hosts": {"web-01", "web-02"},
    "config_path": Path("/etc/app/config.yaml"),
}
print(json.dumps(alert, cls=SREJSONEncoder, indent=2))

# Alternative: use default= parameter
json.dumps(alert, default=str)  # converts everything unknown to str

# ---- JSON PATH-LIKE ACCESS ----

def jq(data: Any, path: str, default=None) -> Any:
    """Simple jq-like path accessor: 'a.b.c' or 'a[0].b'"""
    import re
    parts = re.split(r'\.|\[(\d+)\]', path)
    current = data
    for part in parts:
        if part is None:
            continue
        if not part:
            continue
```

```
        try:
            if part.isdigit():
                current = current[int(part)]
            else:
                current = current[part]
        except (KeyError, IndexError, TypeError):
            return default
    return current

# Usage
pod_data = {"status": {"conditions": [{"type": "Ready", "status":
"True"}]}}
condition_type = jq(pod_data, "status.conditions[0].type")  # "Ready"

# ---- YAML ----

# ⚠ ALWAYS use yaml.safe_load() — yaml.load() can execute arbitrary code
with open("deployment.yaml") as f:
    manifest = yaml.safe_load(f)

# Parse YAML string
yaml_text = """
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
  namespace: production
spec:
  replicas: 3
"""
manifest = yaml.safe_load(yaml_text)

# Multiple documents in one file (separated by ---)
with open("manifests.yaml") as f:
    documents = list(yaml.safe_load_all(f))

# Serialize to YAML
yaml_output = yaml.dump(
    manifest,
    default_flow_style=False,  # use block style (human-readable)
    sort_keys=False,           # preserve insertion order
    allow_unicode=True,
)
print(yaml_output)

# Dump multiple documents
with open("output.yaml", "w") as f:
    yaml.dump_all(documents, f, default_flow_style=False)

# ---- KUBERNETES MANIFEST MANIPULATION ----

def patch_image(manifest_path: str, container_name: str, new_image: str) -
> str:
    """Update container image in a Kubernetes Deployment manifest."""
```

```python
    with open(manifest_path) as f:
        manifest = yaml.safe_load(f)

    containers = (manifest
        .get("spec", {})
        .get("template", {})
        .get("spec", {})
        .get("containers", [])
    )

    for container in containers:
        if container["name"] == container_name:
            old_image = container["image"]
            container["image"] = new_image
            print(f"Updated {container_name}: {old_image} → {new_image}")

    return yaml.dump(manifest, default_flow_style=False)

# ---- TOML (Python 3.11+ stdlib) ----

import tomllib  # Python 3.11+ (read-only in stdlib)

with open("pyproject.toml", "rb") as f:  # must open in binary mode
    config = tomllib.load(f)

# ---- JSON SCHEMA VALIDATION ----

# pip install jsonschema
from jsonschema import validate, ValidationError

DEPLOYMENT_SCHEMA = {
    "type": "object",
    "required": ["service", "version", "replicas"],
    "properties": {
        "service": {"type": "string", "minLength": 1},
        "version": {"type": "string", "pattern": r"^v\d+\.\d+\.\d+$"},
        "replicas": {"type": "integer", "minimum": 1, "maximum": 20},
        "namespace": {"type": "string", "default": "default"},
    }
}

def validate_deployment_config(config: dict) -> None:
    try:
        validate(instance=config, schema=DEPLOYMENT_SCHEMA)
    except ValidationError as e:
        raise ValueError(f"Invalid deployment config: {e.message}") from e
```

💬 Short Interview Answer

"For JSON, the key is the `default=` parameter or a custom encoder to handle non-serializable types like datetime, Path, and sets. Always use `json.dumps(data, indent=2, sort_keys=True)` for human-readable output and git-diffable config files. For YAML, ALWAYS use `yaml.safe_load()` —

`yaml.load()` can execute arbitrary Python code and is a security vulnerability. For Kubernetes manifest manipulation, I load the YAML, modify the Python dict, and dump it back — this is safer than regex-based string manipulation."

⚠ Tricky Gotchas

- **`yaml.load()` is a code execution vulnerability** — `yaml.safe_load()` always
- **YAML "Norway problem"**: `NO` in YAML is parsed as `False`! Use quotes: `"NO"`. Same for `YES`, `ON`, `OFF`, `TRUE`, `FALSE`, `NULL`
- **JSON keys must be strings** — Python dicts with integer keys will be serialized as strings; parsing them back gives strings not ints
- **`json.loads()` vs `json.load()`** — `loads` takes a string, `load` takes a file object

🔗 DevOps Connection

Kubernetes manifests (YAML), Terraform JSON outputs, CloudWatch log events (JSON), GitHub Actions workflow files (YAML), Ansible playbooks (YAML) — everything in DevOps is YAML or JSON. Programmatic manipulation of these formats in Python is a core SRE skill.

---

16. HTTP and REST APIs

📖 What It Is (Simple Terms)

Python's `requests` library is the standard way to make HTTP calls to REST APIs — Kubernetes API server, Prometheus, PagerDuty, Slack, Datadog, GitHub, cloud provider APIs. `httpx` is the modern async alternative.

⚙️ How It Works Internally

`requests` uses `urllib3` under the hood, which manages connection pooling. The `Session` object reuses TCP connections across requests (persistent connections) which is dramatically faster for multiple API calls to the same host.

```
import requests
import json
import time
import logging
from typing import Optional, Any
from urllib.parse import urljoin, urlencode

logger = logging.getLogger(__name__)

# ---- BASIC USAGE ----

# Simple GET
response = requests.get(
    "https://api.example.com/v1/services",
    headers={"Authorization": "Bearer mytoken123"},
    timeout=30,  # ⚠ ALWAYS set timeout — default is no timeout!
```

```python
)
response.raise_for_status()  # raises HTTPError for 4xx/5xx
data = response.json()

# POST with JSON body
response = requests.post(
    "https://api.pagerduty.com/incidents",
    json={  # automatically sets Content-Type: application/json
        "incident": {
            "type": "incident",
            "title": "API Gateway is DOWN",
            "service": {"id": "SERVICE_ID", "type": "service_reference"}
        }
    },
    headers={
        "Authorization": "Token token=API_KEY",
        "Accept": "application/vnd.pagerduty+json;version=2",
    },
    timeout=10,
)

# ---- SESSION FOR PERFORMANCE ----

class KubernetesAPIClient:
    """Client for Kubernetes API with connection pooling and auth."""

    def __init__(
        self,
        base_url: str,
        token: str,
        ca_cert: str | None = None,
        verify_ssl: bool = True,
    ):
        self.base_url = base_url.rstrip("/")
        self.session = requests.Session()
        self.session.headers.update({
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json",
            "Accept": "application/json",
        })
        if ca_cert:
            self.session.verify = ca_cert
        else:
            self.session.verify = verify_ssl

        # Retry adapter
        from requests.adapters import HTTPAdapter
        from urllib3.util.retry import Retry

        retry_strategy = Retry(
            total=3,
            backoff_factor=1,
            status_forcelist=[429, 500, 502, 503, 504],
            allowed_methods=["GET"],  # only retry safe methods
```

```
        )
        adapter = HTTPAdapter(max_retries=retry_strategy)
        self.session.mount("https://", adapter)
        self.session.mount("http://", adapter)

    def get(self, path: str, **kwargs) -> dict:
        url = f"{self.base_url}{path}"
        resp = self.session.get(url, timeout=30, **kwargs)
        resp.raise_for_status()
        return resp.json()

    def post(self, path: str, data: dict, **kwargs) -> dict:
        url = f"{self.base_url}{path}"
        resp = self.session.post(url, json=data, timeout=30, **kwargs)
        resp.raise_for_status()
        return resp.json()

    def patch(self, path: str, data: dict, **kwargs) -> dict:
        url = f"{self.base_url}{path}"
        resp = self.session.patch(url, json=data, timeout=30, **kwargs)
        resp.raise_for_status()
        return resp.json()

    def get_pods(self, namespace: str) -> list[dict]:
        data = self.get(f"/api/v1/namespaces/{namespace}/pods")
        return data.get("items", [])

    def scale_deployment(self, name: str, namespace: str, replicas: int) -
> None:
        self.patch(

f"/apis/apps/v1/namespaces/{namespace}/deployments/{name}/scale",
            {"spec": {"replicas": replicas}}
        )

# ---- PROMETHEUS QUERY CLIENT ----

class PrometheusClient:
    def __init__(self, url: str):
        self.url = url.rstrip("/")
        self.session = requests.Session()

    def query(self, promql: str, time: Optional[float] = None) ->
list[dict]:
        """Execute instant query."""
        params = {"query": promql}
        if time:
            params["time"] = time

        resp = self.session.get(
            f"{self.url}/api/v1/query",
            params=params,
            timeout=30,
        )
```

```
        resp.raise_for_status()
        data = resp.json()

        if data["status"] != "success":
            raise RuntimeError(f"Prometheus query failed:
{data.get('error')}")

        return data["data"]["result"]

    def query_range(
        self, promql: str, start: float, end: float, step: str = "1m"
    ) -> list[dict]:
        """Execute range query."""
        resp = self.session.get(
            f"{self.url}/api/v1/query_range",
            params={"query": promql, "start": start, "end": end, "step":
step},
            timeout=60,
        )
        resp.raise_for_status()
        data = resp.json()
        return data["data"]["result"]

    def get_current_error_rate(self, service: str) -> float:
        """Get current error rate for a service."""
        results = self.query(
            f'sum(rate(http_requests_total{{service="
{service}",status=~"5.."}}[5m])) / '
            f'sum(rate(http_requests_total{{service="{service}"}}[5m]))'
        )
        if not results:
            return 0.0
        return float(results[0]["value"][1])

# ---- WEBHOOK NOTIFICATIONS ----

def send_slack_alert(webhook_url: str, message: str, color: str =
"danger") -> None:
    """Send a Slack alert via incoming webhook."""
    payload = {
        "attachments": [{
            "color": color,  # "good", "warning", "danger" or hex color
            "text": message,
            "footer": "SRE Automation",
            "ts": int(time.time()),
        }]
    }
    response = requests.post(webhook_url, json=payload, timeout=10)
    response.raise_for_status()

def send_pagerduty_alert(routing_key: str, summary: str, severity: str =
"critical") -> str:
    """Trigger a PagerDuty incident and return dedup_key."""
    response = requests.post(
```

```
        "https://events.pagerduty.com/v2/enqueue",
        json={
            "routing_key": routing_key,
            "event_action": "trigger",
            "payload": {
                "summary": summary,
                "severity": severity,
                "source": "sre-automation",
                "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ",
time.gmtime()),
            }
        },
        timeout=10,
    )
    response.raise_for_status()
    return response.json()["dedup_key"]

# ---- PAGINATION HANDLING ----

def get_all_pages(url: str, headers: dict, page_size: int = 100) -> list:
    """Handle paginated API responses."""
    results = []
    page = 1

    while True:
        resp = requests.get(
            url,
            params={"page": page, "per_page": page_size},
            headers=headers,
            timeout=30,
        )
        resp.raise_for_status()
        data = resp.json()

        items = data.get("items", data if isinstance(data, list) else [])
        results.extend(items)

        # Check for more pages (varies by API)
        if len(items) < page_size:  # got fewer than max — last page
            break
        if "next" not in resp.links:  # GitHub-style Link header
            break

        page += 1

    return results
```

💬 Short Interview Answer

"The two most important rules for HTTP clients in SRE scripts: always set a timeout (the default in requests is no timeout, meaning your script can hang forever waiting for a slow API), and always use a Session object when making multiple calls to the same host — it reuses the TCP connection and is much faster. For

resilience, attach a `Retry` adapter with `status_forcelist=[429, 503, 504]` to automatically retry on transient errors, but only for safe/idempotent methods like GET. Use `raise_for_status()` to automatically raise exceptions on 4xx/5xx instead of manually checking status codes."

⚠ Tricky Gotchas

- **No default timeout** — `requests.get(url)` can hang forever; ALWAYS use `timeout=(connect_timeout, read_timeout)`
- **`requests.get(url, json=data)` sets the body** but GET requests shouldn't have bodies — use `params=` for query parameters
- **`response.json()` raises if response body is not valid JSON** — even for 200 OK responses
- **SSL verification** — `verify=False` disables SSL checking; never do this in production; use `verify="/path/to/ca.pem"` for custom CAs
- **Connection pooling**: Session re-uses connections but there's a default pool size of 10 — for high-concurrency use, increase it with `HTTPAdapter(pool_connections=100, pool_maxsize=100)`

🔗 DevOps Connection

Prometheus (metrics queries), PagerDuty (alerting), Slack (notifications), GitHub API (PR automation), ArgoCD API (deployment status), Datadog API (dashboards, monitors) — all consumed via HTTP REST APIs from SRE scripts.

---

17. Concurrency

📖 What It Is (Simple Terms)

Python has three concurrency models: **threading** (shared memory, GIL-limited for CPU tasks, good for I/O), **multiprocessing** (true parallelism, separate memory spaces), and **asyncio** (cooperative multitasking, best for I/O-heavy code like checking many endpoints concurrently).

⚙️ How It Works Internally

- **GIL (Global Interpreter Lock)**: CPython only executes one thread's Python bytecode at a time. Threading works well for I/O (GIL is released during I/O syscalls) but doesn't speed up CPU-bound code.
- **asyncio**: Single-threaded event loop. `await` yields control to the event loop which can run another coroutine while waiting for I/O.
- **multiprocessing**: Separate Python interpreter processes — no GIL sharing. Great for CPU-bound tasks.

```
import threading
import multiprocessing
import asyncio
import concurrent.futures
import time
from typing import Callable, Any
```

```python
# ---- THREADING (I/O-bound: health checks, API calls) ----

import requests
from threading import Thread, Lock, Event

results = {}
lock = Lock()

def check_health(host: str) -> None:
    try:
        resp = requests.get(f"http://{host}/health", timeout=5)
        healthy = resp.status_code == 200
    except requests.RequestException:
        healthy = False

    with lock:  # thread-safe dict update
        results[host] = healthy

hosts = ["api-01:8080", "api-02:8080", "api-03:8080", "api-04:8080"]

threads = [Thread(target=check_health, args=(host,)) for host in hosts]
for t in threads:
    t.start()
for t in threads:
    t.join(timeout=10)  # wait up to 10s per thread

unhealthy = [h for h, ok in results.items() if not ok]

# ---- THREADPOOLEXECUTOR (cleaner threading interface) ----

from concurrent.futures import ThreadPoolExecutor, as_completed, wait

def check_endpoint(url: str) -> tuple[str, bool, float]:
    start = time.time()
    try:
        resp = requests.get(url, timeout=5)
        return url, resp.status_code == 200, time.time() - start
    except Exception:
        return url, False, time.time() - start

urls = [f"https://service-{i}.internal/health" for i in range(1, 21)]

with ThreadPoolExecutor(max_workers=10) as executor:
    futures = {executor.submit(check_endpoint, url): url for url in urls}

    for future in as_completed(futures, timeout=30):
        url, healthy, latency = future.result()
        if not healthy:
            print(f"UNHEALTHY: {url} ({latency:.2f}s)")

# ---- ASYNCIO (best for many concurrent I/O operations) ----

import aiohttp
import asyncio
```

```
async def check_health_async(session: aiohttp.ClientSession, url: str) ->
tuple[str, bool]:
    try:
        async with session.get(url,
timeout=aiohttp.ClientTimeout(total=5)) as resp:
            return url, resp.status == 200
    except Exception:
        return url, False

async def check_all_async(urls: list[str]) -> dict[str, bool]:
    async with aiohttp.ClientSession() as session:
        tasks = [check_health_async(session, url) for url in urls]
        results = await asyncio.gather(*tasks, return_exceptions=True)

    return {
        url: healthy
        for result in results
        if not isinstance(result, Exception)
        for url, healthy in [result]
    }

# Run async from sync context
results = asyncio.run(check_all_async(urls))

# ---- ASYNCIO WITH SEMAPHORE (rate limiting) ----

async def check_all_with_limit(urls: list[str], concurrency: int = 20) ->
dict[str, bool]:
    """Check URLs with max concurrency limit."""
    semaphore = asyncio.Semaphore(concurrency)

    async def bounded_check(session, url):
        async with semaphore:  # limits to `concurrency` simultaneous
requests
            return await check_health_async(session, url)

    async with aiohttp.ClientSession() as session:
        tasks = [bounded_check(session, url) for url in urls]
        results = await asyncio.gather(*tasks, return_exceptions=True)

    return {url: healthy for url, healthy in results if not
isinstance(healthy, Exception)}

# ---- ASYNCIO TASK GROUPS (Python 3.11+) ----

async def deploy_all(services: list[str]) -> None:
    async with asyncio.TaskGroup() as tg:
        tasks = {svc: tg.create_task(deploy_async(svc)) for svc in
services}
    # All tasks complete (or first exception propagates)

# ---- MULTIPROCESSING (CPU-bound: log processing, compression) ----
```

```python
import multiprocessing as mp
from concurrent.futures import ProcessPoolExecutor

def process_log_chunk(lines: list[str]) -> dict:
    """CPU-intensive log analysis — runs in separate process."""
    from collections import Counter
    import re

    pattern = re.compile(r'(ERROR|WARN|INFO|DEBUG)')
    counts = Counter()
    for line in lines:
        if m := pattern.search(line):
            counts[m.group(1)] += 1
    return dict(counts)

def analyze_logs_parallel(log_file: str) -> dict:
    """Process large log file in parallel chunks."""
    with open(log_file) as f:
        all_lines = f.readlines()

    chunk_size = len(all_lines) // mp.cpu_count()
    chunks = [all_lines[i:i+chunk_size] for i in range(0, len(all_lines),
chunk_size)]

    from collections import Counter
    total = Counter()

    with ProcessPoolExecutor(max_workers=mp.cpu_count()) as executor:
        for chunk_result in executor.map(process_log_chunk, chunks):
            total.update(chunk_result)

    return dict(total)

# ---- CHOOSING THE RIGHT MODEL ----
# I/O-bound (API calls, DB queries, file reads) → asyncio or
ThreadPoolExecutor
# CPU-bound (parsing, encryption, compression) → ProcessPoolExecutor
# Mixed workload with callbacks → ThreadPoolExecutor
# Simple background task → Thread (daemon=True)
```

💬 Short Interview Answer

"For SRE scripts checking 100+ endpoints simultaneously, asyncio with `asyncio.gather()` is the best choice — single thread, no lock management, very low overhead. For wrapping blocking library calls (like boto3 which isn't async-native), ThreadPoolExecutor is cleaner. The GIL means threading doesn't help with CPU-bound work — use ProcessPoolExecutor for that. Key asyncio concept: always use a Semaphore to limit concurrency when calling external APIs to avoid rate limiting. In Python 3.11+, `asyncio.TaskGroup` gives you structured concurrency with proper error propagation."

⚠ Tricky Gotchas

- **`asyncio.run()` creates and tears down a new event loop** — don't nest `asyncio.run()` calls

- `threading.Thread(daemon=True)` — daemon threads are killed when main thread exits; non-daemon threads keep the process alive
- **Shared mutable state between threads** needs Lock; between processes needs `multiprocessing.Manager` or queues
- `concurrent.futures.as_completed()` **timeout** is total timeout across all futures, not per future
- `asyncio.gather(*tasks, return_exceptions=True)` — without this, first exception cancels all others

🔗 DevOps Connection

Mass health checking (checking all 500 pods simultaneously), parallel AWS API calls (describing all EC2 instances across all regions), batch Prometheus queries — all benefit from concurrent execution. Without concurrency, a script checking 100 endpoints at 1s each takes 100 seconds sequentially; with asyncio it takes ~1-2 seconds.

---

18. Regular Expressions

📖 What It Is (Simple Terms)

Regular expressions are patterns for matching, searching, and extracting text. For SRE work, they're essential for log parsing, config validation, URL pattern matching, and extracting metrics from text output.

🧩 Key Patterns

```
import re

# ---- COMPILE PATTERNS (always compile when used in loops) ----

# Basic patterns
IP_RE = re.compile(r'\b(\d{1,3}\.){3}\d{1,3}\b')
K8S_POD_RE = re.compile(r'(?P<deployment>[a-z0-9-]+)-(?P<rs>[a-z0-9]
{5,10})-(?P<pod>[a-z0-9]{5})')
LOG_RE = re.compile(
    r'^(?P<timestamp>\d{4}-\d{2}-\d{2}[T ]\d{2}:\d{2}:\d{2}(?:\.\d+)?(?:Z|
[+-]\d{2}:\d{2})?)\s+'
    r'(?P<level>TRACE|DEBUG|INFO|WARN(?:ING)?|ERROR|CRITICAL|FATAL)\s+'
    r'(?P<message>.+)$',
    re.IGNORECASE
)

SEMVER_RE = re.compile(
    r'^v?(?P<major>\d+)\.(?P<minor>\d+)\.(?P<patch>\d+)'
    r'(?:-(?P<prerelease>[0-9A-Za-z.-]+))?'
    r'(?:\+(?P<buildmeta>[0-9A-Za-z.-]+))?$'
)

# ---- METHODS ----

text = "2024-01-15T10:23:45Z ERROR web-01 Connection refused to db:5432"
```

```
# re.match() — anchored at start
# re.search() — anywhere in string
# re.fullmatch() — entire string must match
# re.findall() — return all non-overlapping matches as list of strings
# re.finditer() — return iterator of Match objects (memory efficient)
# re.sub() — replace matches
# re.split() — split on pattern

# search
m = LOG_RE.search(text)
if m:
    level = m.group("level")       # "ERROR"
    ts = m.group("timestamp")      # "2024-01-15T10:23:45Z"
    msg = m.group("message")       # "web-01 Connection refused..."
    print(m.groupdict())           # dict of all named groups

# findall — returns list of strings (or list of tuples for multiple
groups)
ips = IP_RE.findall("Source: 10.0.1.5, Dest: 192.168.1.100")
# ['10.0.1.5', '192.168.1.100']

# finditer — memory efficient for large files
with open("/var/log/app.log") as f:
    for line in f:
        for match in K8S_POD_RE.finditer(line):
            deployment = match.group("deployment")

# sub — replacement
# Mask sensitive data
CREDIT_CARD_RE = re.compile(r'\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b')
sanitized = CREDIT_CARD_RE.sub(r'****-****-****-****', log_line)

# sub with function
def increment_version(match):
    parts = match.group().lstrip('v').split('.')
    parts[-1] = str(int(parts[-1]) + 1)
    return 'v' + '.'.join(parts)

new_version = re.sub(r'v\d+\.\d+\.\d+', increment_version, "image:
myapp:v1.2.3")

# split with capturing group (keeps delimiter)
parts = re.split(r'(\s+)', "  hello   world  ")
# ['', '  ', 'hello', '   ', 'world', '  ', '']

# ---- COMMON SRE PATTERNS ----

# Validate Kubernetes resource name
K8S_NAME_RE = re.compile(r'^[a-z0-9][a-z0-9-]{0,61}[a-z0-9]$')
def is_valid_k8s_name(name: str) -> bool:
    return bool(K8S_NAME_RE.match(name))

# Extract metrics from text (e.g., from top, df, vmstat)
DF_RE = re.compile(r'(?P<fs>\S+)\s+(?P<size>\d+)\s+(?P<used>\d+)\s+(?
```

```
P<avail>\d+)\s+(?P<pct>\d+)%\s+(?P<mount>\S+)')

def parse_df_output(df_output: str) -> list[dict]:
    results = []
    for line in df_output.strip().split('\n')[1:]:  # skip header
        if m := DF_RE.match(line.strip()):
            results.append({
                "filesystem": m.group("fs"),
                "used_pct": int(m.group("pct")),
                "mount": m.group("mount"),
            })
    return results

# Extract HTTP status codes from nginx access logs
NGINX_RE = re.compile(
    r'(?P<ip>\S+) \S+ \S+ \[(?P<time>[^\]]+)\] '
    r'"(?P<method>\w+) (?P<path>\S+) HTTP/[\d.]+" '
    r'(?P<status>\d{3}) (?P<bytes>\d+)'
)

# ---- FLAGS ----

re.IGNORECASE    # case-insensitive (re.I)
re.MULTILINE     # ^ and $ match start/end of each line (re.M)
re.DOTALL        # . matches \n too (re.S)
re.VERBOSE       # allows whitespace and comments in pattern (re.X)

# Verbose mode for complex patterns
VERSION_RE = re.compile(r"""
    ^v?                     # optional v prefix
    (?P<major>\d+)          # major version
    \.                      # dot separator
    (?P<minor>\d+)          # minor version
    \.                      # dot separator
    (?P<patch>\d+)          # patch version
    (?:-(?P<pre>[a-z0-9.]+))?  # optional pre-release
    $
""", re.VERBOSE | re.IGNORECASE)
```

💬 Short Interview Answer

"For SRE work, always compile regex patterns with `re.compile()` when they're used in loops — compilation is expensive and caching it gives 10-20x speedup. Always use named groups for readability: `(?P<level>ERROR|WARN)` is self-documenting. Use `re.VERBOSE` for complex patterns so you can add comments. For large log files, use `re.finditer()` not `re.findall()` — finditer returns an iterator and doesn't build the entire result list in memory."

⚠ Tricky Gotchas

- **`re.match()` only matches at start of string** — most SRE use cases need `re.search()`

- **Greedy vs lazy quantifiers**: `.*` is greedy (matches as much as possible); `.*?` is lazy (matches as little as possible) — matters when parsing nested structures
- **`re.sub()` with backreferences**: `r'\1'` references first capture group — use `r'\g<name>'` for named groups
- **Catastrophic backtracking**: nested quantifiers like `(a+)+` can cause exponential time on adversarial input — relevant for user-supplied patterns

🔗 DevOps Connection

Log parsing, nginx/haproxy log analysis, Kubernetes pod name parsing, config file validation, secret detection (scanning for leaked credentials in code), URL pattern routing — all heavily regex-based.

🟠 ADVANCED TOPICS

---

19. Context Managers

📖 What It Is (Simple Terms)

Context managers implement the `with` statement, guaranteeing that setup and teardown code runs in pairs — even if an exception occurs. Beyond file handles, you can write your own for database connections, locks, timers, temporary directories, and any resource that needs cleanup.

⚙️ How It Works Internally

The `with` statement calls `__enter__()` and stores the return value (what `as` binds to), then calls `__exit__(exc_type, exc_val, exc_tb)` on exit. If `__exit__` returns a truthy value, the exception is **suppressed**. `contextlib.contextmanager` wraps a generator function — code before `yield` is `__enter__`, code after is `__exit__`.

```
from contextlib import contextmanager, asynccontextmanager, suppress,
nullcontext
import time
import logging
import threading
import tempfile
import os
from pathlib import Path
from typing import Generator

logger = logging.getLogger(__name__)

# ---- CLASS-BASED CONTEXT MANAGER ----

class DatabaseConnection:
    def __init__(self, dsn: str):
        self.dsn = dsn
        self.conn = None
```

```python
    def __enter__(self):
        logger.info(f"Connecting to {self.dsn}")
        self.conn = connect(self.dsn)
        return self.conn  # this is the `as` value

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.conn:
            if exc_type:  # exception occurred
                self.conn.rollback()
                logger.error(f"Transaction rolled back due to: {exc_val}")
            else:
                self.conn.commit()
            self.conn.close()
        return False  # False = don't suppress exceptions

with DatabaseConnection("postgresql://localhost/mydb") as conn:
    conn.execute("INSERT INTO events ...")

# ---- GENERATOR-BASED CONTEXT MANAGER ----

@contextmanager
def timer(operation: str) -> Generator[None, None, None]:
    """Time a block of code and log the duration."""
    start = time.perf_counter()
    try:
        yield  # control passes to the with block
    finally:
        elapsed = time.perf_counter() - start
        logger.info(f"{operation} completed in {elapsed:.3f}s")

with timer("database migration"):
    run_migrations()

@contextmanager
def temp_kubeconfig(cluster_config: dict):
    """Write a kubeconfig to a temp file and set KUBECONFIG env var."""
    import yaml
    tmp = tempfile.NamedTemporaryFile(
        mode='w', suffix='.yaml', delete=False
    )
    try:
        yaml.dump(cluster_config, tmp)
        tmp.flush()
        tmp.close()
        old_kube = os.environ.get("KUBECONFIG")
        os.environ["KUBECONFIG"] = tmp.name
        yield tmp.name
    finally:
        os.environ.pop("KUBECONFIG", None)
        if old_kube:
            os.environ["KUBECONFIG"] = old_kube
        Path(tmp.name).unlink(missing_ok=True)
```

```
with temp_kubeconfig(prod_cluster_config):
    subprocess.run(["kubectl", "get", "pods", "-n", "production"])

@contextmanager
def managed_lock(lock_file: str, timeout: float = 30):
    """File-based distributed lock with timeout."""
    deadline = time.time() + timeout
    lock_path = Path(lock_file)

    while True:
        try:
            fd = open(lock_path, 'x')  # exclusive create
            break
        except FileExistsError:
            if time.time() > deadline:
                raise TimeoutError(f"Could not acquire lock {lock_file}
within {timeout}s")
            time.sleep(0.5)

    try:
        fd.write(str(os.getpid()))
        fd.close()
        yield lock_path
    finally:
        lock_path.unlink(missing_ok=True)

with managed_lock("/tmp/deploy.lock"):
    run_deployment()

# ---- ASYNC CONTEXT MANAGER ----

@asynccontextmanager
async def managed_http_session():
    """Async HTTP session with automatic cleanup."""
    import aiohttp
    async with aiohttp.ClientSession() as session:
        yield session

async def run():
    async with managed_http_session() as session:
        async with session.get("https://api.example.com") as resp:
            data = await resp.json()

# ---- CONTEXTLIB UTILITIES ----

# suppress — cleanly ignore specific exceptions
with suppress(FileNotFoundError, PermissionError):
    os.unlink("/tmp/stale_file")

# nullcontext — conditional context manager
def deploy(service, dry_run=False):
    ctx = managed_lock("/tmp/deploy.lock") if not dry_run else
nullcontext()
    with ctx:
```

```
        do_deployment(service)

# ExitStack — dynamic context managers
from contextlib import ExitStack

def deploy_to_multiple_clusters(clusters: list[dict]) -> None:
    """Deploy to multiple clusters, each with its own kubeconfig."""
    with ExitStack() as stack:
        for cluster in clusters:
            # Dynamically enter context managers
            stack.enter_context(temp_kubeconfig(cluster))
        # All kubeconfigs are active here
        for cluster in clusters:
            deploy_to_cluster(cluster["name"])
    # All kubeconfigs cleaned up here
```

💬 Short Interview Answer

"Context managers are one of Python's most powerful patterns for SRE tooling. The generator-based `@contextmanager` decorator is the cleanest way — code before `yield` is setup, code after (in `finally`) is teardown. I use them for: file locks (deploy locks), temporary kubeconfig files, operation timing for SLO measurements, database transaction management, and anywhere I need guaranteed cleanup. The `ExitStack` from contextlib is underused but essential when you need to dynamically compose multiple context managers — like setting up kubeconfigs for N clusters where N isn't known at compile time."

⚠ Tricky Gotchas

- **`__exit__` returning `True` suppresses the exception** — usually you want `return False`
- **`@contextmanager` must have exactly one `yield`** — two yields raises `RuntimeError`
- **Exception handling in generators**: in `@contextmanager`, if an exception occurs in the `with` block, it's thrown INTO the generator at the `yield` point — wrap `yield` in try/except/finally appropriately
- **`suppress` doesn't work with `KeyboardInterrupt` by default** — `KeyboardInterrupt` inherits from `BaseException` not `Exception`

🔗 DevOps Connection

Database transaction management, distributed deploy locks (prevent parallel deploys), temporary credential files, kubectl context switching, AWS role assumption, connection pool management — all modeled cleanly as context managers.

---

20. Generators and Iterators

📖 What It Is (Simple Terms)

Generators are functions that `yield` values one at a time instead of returning a full list. They're **lazy** — they compute values on demand. For SRE work, they're critical for processing large log files, streaming API responses, and building memory-efficient data pipelines.

⚙ How It Works Internally

A generator function returns a **generator object** (an iterator) when called. Calling `next()` on it runs the function body until the next `yield`, then suspends. State (local variables) is preserved between calls. When the function returns, `StopIteration` is raised.

```
from typing import Generator, Iterator
import itertools

# ---- BASIC GENERATOR ----

def count_errors_in_file(filepath: str) -> Generator[dict, None, None]:
    """Stream error log entries without loading entire file."""
    with open(filepath) as f:
        for i, line in enumerate(f, 1):
            if "ERROR" in line:
                yield {
                    "line_num": i,
                    "content": line.rstrip(),
                    "timestamp": extract_timestamp(line),
                }

# Memory efficient: processes one line at a time regardless of file size
for error in count_errors_in_file("/var/log/huge-app.log"):
    alert_if_critical(error)

# ---- GENERATOR PIPELINES ----

def read_lines(filepath: str) -> Iterator[str]:
    with open(filepath) as f:
        yield from f  # yield each line

def filter_errors(lines: Iterator[str]) -> Iterator[str]:
    return (line for line in lines if "ERROR" in line)

def parse_log_line(lines: Iterator[str]) -> Iterator[dict]:
    for line in lines:
        parts = line.strip().split(None, 3)
        if len(parts) >= 4:
            yield {"ts": parts[0], "level": parts[1], "host": parts[2],
"msg": parts[3]}

def enrich_with_location(entries: Iterator[dict]) -> Iterator[dict]:
    import socket
    cache = {}
    for entry in entries:
        host = entry.get("host", "")
        if host not in cache:
            try:
                cache[host] = socket.gethostbyname(host)
            except socket.gaierror:
                cache[host] = "unknown"
        entry["ip"] = cache[host]
        yield entry
```

```python
# Compose the pipeline — all lazy, no intermediate lists
pipeline = enrich_with_location(
    parse_log_line(
        filter_errors(
            read_lines("/var/log/app.log")
        )
    )
)

# Only now does execution start; processes one line at a time
for entry in itertools.islice(pipeline, 100):  # first 100 errors
    print(entry)

# ---- YIELD FROM ----

def merge_log_files(*filepaths: str) -> Iterator[str]:
    """Merge multiple log files."""
    for filepath in filepaths:
        yield from read_lines(filepath)

# ---- SEND/THROW (two-way communication) ----

def metric_accumulator() -> Generator[None, float, dict]:
    """Coroutine that accumulates metrics."""
    values = []
    while True:
        value = yield  # receive values via .send()
        if value is None:
            break
        values.append(value)
    return {
        "count": len(values),
        "mean": sum(values) / len(values) if values else 0,
        "min": min(values) if values else 0,
        "max": max(values) if values else 0,
    }

acc = metric_accumulator()
next(acc)  # prime the generator (advance to first yield)
acc.send(12.4)
acc.send(15.7)
acc.send(8.9)
try:
    acc.send(None)  # signal end
except StopIteration as e:
    stats = e.value  # get the return value
    print(stats)

# ---- ITERTOOLS FOR SRE PATTERNS ----

import itertools

# islice — take first N items from any iterator
```

```python
first_100_errors = list(itertools.islice(error_generator, 100))

# chain — concatenate multiple iterables
all_logs = itertools.chain(
    read_lines("/var/log/app.log"),
    read_lines("/var/log/app.log.1"),
    read_lines("/var/log/app.log.2"),
)

# groupby — group consecutive equal elements
import operator
logs = sorted(parsed_logs, key=operator.itemgetter("level"))
for level, group in itertools.groupby(logs,
key=operator.itemgetter("level")):
    entries = list(group)
    print(f"{level}: {len(entries)} entries")

# batched (Python 3.12+) or manual chunking
def batched(iterable, n: int):
    """Yield successive n-sized chunks from iterable."""
    it = iter(iterable)
    while True:
        chunk = list(itertools.islice(it, n))
        if not chunk:
            return
        yield chunk

# Process in batches of 100 for Datadog metrics API
for batch in batched(metric_points, 100):
    post_metrics_batch(batch)

# takewhile / dropwhile — conditional iteration
def tail_until_silence(log_file: str, timeout: float = 5.0):
    """Yield log lines, stop if no new lines for `timeout` seconds."""
    import time
    last_line_time = time.time()

    with open(log_file) as f:
        f.seek(0, 2)  # start at end
        while time.time() - last_line_time < timeout:
            line = f.readline()
            if line:
                last_line_time = time.time()
                yield line.rstrip()
            else:
                time.sleep(0.1)
```

💬 Short Interview Answer

"Generators are essential for SRE scripts that process large volumes of data — log files, API response streams, metric time series. The key mental model is 'pipeline': each generator is a lazy transformation stage that processes one item at a time. A 10GB log file doesn't need to fit in memory — you chain

`read_lines()`, `filter_errors()`, `parse_entries()` generators and only one line is in memory at a time. `yield from` is the clean way to delegate to sub-generators. `itertools.islice()`, `batched()`, and `chain()` are the most useful SRE-relevant itertools."

⚠ Tricky Gotchas

- **Generators are exhausted after one iteration** — you can't iterate a generator twice; store in `list()` if you need to
- **`yield from` vs `for x in gen: yield x`** — they're equivalent but `yield from` is faster and handles `send()` and `throw()` correctly
- **Generator expressions `(x for x in ...)` are lazy** but list comprehensions `[x for x in ...]` are eager
- **`StopIteration` inside a generator in Python 3.7+ is converted to `RuntimeError`** — don't catch `StopIteration` to signal end-of-generator inside a generator

🔗 DevOps Connection

Log file processing pipelines, metric ingestion streams, Kubernetes event watching (watch API streams events), batch processing of infrastructure inventory — all benefit from lazy generator pipelines that avoid loading entire datasets into memory.

---

21. Decorators — Advanced

📖 What It Is (Simple Terms)

Decorators are functions that wrap other functions to add behavior — logging, timing, caching, rate limiting, retry, authentication — without modifying the original function's code. They're the Python implementation of the Aspect-Oriented Programming pattern.

🧩 Key Patterns

```
import time
import logging
import functools
import threading
from typing import Callable, TypeVar, Any, Optional

F = TypeVar("F", bound=Callable[..., Any])
logger = logging.getLogger(__name__)

# ---- BASIC DECORATOR PATTERN ----

def log_calls(func: F) -> F:
    """Log function entry, exit, and duration."""
    @functools.wraps(func)  # ⚠ ALWAYS use @wraps
    def wrapper(*args, **kwargs):
        logger.debug(f"Calling {func.__name__}({args}, {kwargs})")
        start = time.perf_counter()
        try:
```

```
            result = func(*args, **kwargs)
            elapsed = time.perf_counter() - start
            logger.debug(f"{func.__name__} returned in {elapsed:.3f}s")
            return result
        except Exception as e:
            elapsed = time.perf_counter() - start
            logger.error(f"{func.__name__} raised {type(e).__name__} after
{elapsed:.3f}s: {e}")
            raise
    return wrapper

# ---- DECORATOR WITH ARGUMENTS (factory pattern) ----

def rate_limit(calls_per_second: float):
    """Limit function call rate."""
    min_interval = 1.0 / calls_per_second
    last_call = [0.0]  # use list to allow mutation in closure
    lock = threading.Lock()

    def decorator(func: F) -> F:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            with lock:
                elapsed = time.time() - last_call[0]
                if elapsed < min_interval:
                    time.sleep(min_interval - elapsed)
                last_call[0] = time.time()
            return func(*args, **kwargs)
        return wrapper
    return decorator

@rate_limit(calls_per_second=10)
def call_datadog_api(metric: str) -> dict:
    pass

# ---- TIMEOUT DECORATOR ----

import signal
from contextlib import contextmanager

def timeout(seconds: float):
    """Raise TimeoutError if function takes too long (Unix only)."""
    def decorator(func: F) -> F:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            def handler(signum, frame):
                raise TimeoutError(f"{func.__name__} timed out after
{seconds}s")
            old = signal.signal(signal.SIGALRM, handler)
            signal.setitimer(signal.ITIMER_REAL, seconds)
            try:
                return func(*args, **kwargs)
            finally:
                signal.setitimer(signal.ITIMER_REAL, 0)
```

```
                signal.signal(signal.SIGALRM, old)
        return wrapper
    return decorator

@timeout(30)
def slow_health_check(url: str) -> bool:
    pass

# ---- CIRCUIT BREAKER ----

from enum import Enum
from dataclasses import dataclass, field

class CircuitState(Enum):
    CLOSED = "closed"      # normal operation
    OPEN = "open"          # failing, reject all calls
    HALF_OPEN = "half_open" # testing if service recovered

@dataclass
class CircuitBreaker:
    failure_threshold: int = 5
    recovery_timeout: float = 60.0
    success_threshold: int = 2

    state: CircuitState = field(default=CircuitState.CLOSED, init=False)
    failure_count: int = field(default=0, init=False)
    success_count: int = field(default=0, init=False)
    last_failure_time: float = field(default=0.0, init=False)

    def __call__(self, func: F) -> F:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            return self._call(func, *args, **kwargs)
        return wrapper

    def _call(self, func, *args, **kwargs):
        if self.state == CircuitState.OPEN:
            if time.time() - self.last_failure_time >=
self.recovery_timeout:
                self.state = CircuitState.HALF_OPEN
                self.success_count = 0
            else:
                raise RuntimeError(f"Circuit breaker OPEN for
{func.__name__}")

        try:
            result = func(*args, **kwargs)
            self._on_success()
            return result
        except Exception as e:
            self._on_failure()
            raise

    def _on_success(self):
```

```
        if self.state == CircuitState.HALF_OPEN:
            self.success_count += 1
            if self.success_count >= self.success_threshold:
                self.state = CircuitState.CLOSED
                self.failure_count = 0
        else:
            self.failure_count = 0

    def _on_failure(self):
        self.failure_count += 1
        self.last_failure_time = time.time()
        if self.failure_count >= self.failure_threshold:
            self.state = CircuitState.OPEN
            logger.error(f"Circuit breaker opened after
{self.failure_count} failures")

db_circuit = CircuitBreaker(failure_threshold=3, recovery_timeout=30)

@db_circuit
def query_database(sql: str) -> list:
    pass

# ---- LRU CACHE ----

from functools import lru_cache, cache

@lru_cache(maxsize=128)
def get_service_endpoint(service_name: str, env: str) -> str:
    """Cache service discovery lookups."""
    return discover_service(service_name, env)

# Clear cache when services change
get_service_endpoint.cache_clear()

# ---- CLASS DECORATOR ----

def singleton(cls):
    """Make a class a singleton."""
    instances = {}
    @functools.wraps(cls)
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class KubernetesClient:
    def __init__(self):
        self.client = kubernetes.client.CoreV1Api()
```

💬 Short Interview Answer

"Decorators are how Python implements cross-cutting concerns — behaviors that apply across many functions like retry, logging, rate limiting, and circuit breaking. The three-level nesting for parameterized decorators (decorator factory → decorator → wrapper) is the key pattern. Always use `@functools.wraps(func)` in the wrapper — without it, `func.__name__` and `__doc__` are lost, which breaks logging, debugging, and pytest. The circuit breaker pattern implemented as a callable class decorator is a real production pattern used in SRE tooling."

⚠ Tricky Gotchas

- **Forgetting `@wraps`** — the wrapped function loses its name and docstring, confusing logs and debuggers
- **Decorators with state** (like circuit breaker) — the state is shared across all calls; make sure that's intentional
- **Stacking decorators**: `@dec1 @dec2 def f()` applies bottom-up: `dec1(dec2(f))` — the outer decorator wraps the already-wrapped function
- **`@lru_cache` makes arguments hashable-required** — can't use with dict or list arguments

🔗 DevOps Connection

Retry decorators on all external API calls, rate limiting on Datadog/PagerDuty API callers, circuit breakers protecting database calls, `@lru_cache` for service discovery lookups, `@log_calls` for audit trails of deployment operations.

---

22. Dataclasses and Pydantic

📖 What It Is (Simple Terms)

`dataclasses` (Python 3.7+ stdlib) and `pydantic` (third-party, widely used) provide clean ways to define structured data with type hints. Pydantic adds validation — ideal for parsing and validating config, API responses, and event payloads.

🧩 Key Patterns

```python
from dataclasses import dataclass, field, asdict, astuple
from typing import Optional, List
import json

# ---- DATACLASSES ----

@dataclass
class ServiceConfig:
    name: str
    namespace: str
    replicas: int
    image: str
    port: int = 8080
    env_vars: dict = field(default_factory=dict)  # mutable default
    labels: list = field(default_factory=list)
```

```
    # Computed field (not included in __init__)
    healthy: bool = field(default=False, init=False, repr=False)

    def __post_init__(self):
        """Validation and computed fields."""
        if self.replicas < 1:
            raise ValueError(f"replicas must be >= 1, got
{self.replicas}")
        if not self.name.replace("-", "").replace(".", "").isalnum():
            raise ValueError(f"Invalid service name: {self.name}")
        # Computed
        self.healthy = self.replicas > 0

@dataclass(frozen=True)  # immutable (hashable, can be dict key or set
member)
class ServiceEndpoint:
    host: str
    port: int
    protocol: str = "https"

    @property
    def url(self) -> str:
        return f"{self.protocol}://{self.host}:{self.port}"

# Serialize/deserialize
config = ServiceConfig("api-gateway", "production", 3, "myrepo/api:v1.2")
config_dict = asdict(config)
config_json = json.dumps(config_dict)
restored = ServiceConfig(**json.loads(config_json))

# ---- PYDANTIC V2 (validation, serialization) ----

# pip install pydantic
from pydantic import BaseModel, Field, field_validator, model_validator,
ConfigDict
from pydantic import HttpUrl, IPvAnyAddress
from typing import Annotated
from datetime import datetime

class AlertEvent(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True,  # auto-strip strings
        frozen=False,
        extra="ignore",  # ignore unknown fields from API
    )

    alert_id: str = Field(..., min_length=1, description="Unique alert
ID")
    service: str = Field(..., pattern=r'^[a-z0-9][a-z0-9-]*$')
    severity: str = Field(..., pattern=r'^(info|warning|critical|page)$')
    message: str = Field(..., max_length=1000)
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    labels: dict[str, str] = Field(default_factory=dict)
    runbook_url: Optional[HttpUrl] = None
```

```
    @field_validator("severity", mode="before")
    @classmethod
    def normalize_severity(cls, v: str) -> str:
        return v.lower()

    @model_validator(mode="after")
    def validate_critical_has_runbook(self) -> "AlertEvent":
        if self.severity in ("critical", "page") and not self.runbook_url:
            raise ValueError("Critical/page alerts must have a
runbook_url")
        return self

    def to_pagerduty_payload(self) -> dict:
        return {
            "routing_key": "...",
            "event_action": "trigger",
            "payload": {
                "summary": self.message,
                "severity": self.severity,
                "custom_details": {
                    "service": self.service,
                    "labels": self.labels,
                }
            }
        }

# Parsing from dict (e.g., webhook payload, API response)
try:
    event = AlertEvent(**webhook_payload)
except ValidationError as e:
    print(e.json(indent=2))  # structured error messages

# ---- PYDANTIC SETTINGS (Config from env vars) ----

from pydantic_settings import BaseSettings

class AppSettings(BaseSettings):
    model_config = ConfigDict(env_file=".env", case_sensitive=False)

    database_url: str
    redis_url: str = "redis://localhost:6379"
    log_level: str = "INFO"
    max_retries: int = 3
    debug: bool = False
    allowed_origins: list[str] = Field(default_factory=list)

    @field_validator("log_level")
    @classmethod
    def validate_log_level(cls, v):
        valid = {"DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"}
        if v.upper() not in valid:
            raise ValueError(f"log_level must be one of {valid}")
        return v.upper()
```

```
settings = AppSettings()  # reads from env vars + .env file
```

💬 Short Interview Answer

"Dataclasses are for lightweight data containers — they auto-generate `__init__`, `__repr__`, `__eq__`. `frozen=True` makes them immutable and hashable. Pydantic is for data that needs validation — API payloads, config files, event streams. Pydantic V2 uses model validators and field validators that run automatically on construction and give structured error messages. In production SRE tooling, I use Pydantic for all external data (webhook payloads, API responses) because it fails fast with clear errors rather than silently passing bad data through."

⚠ Tricky Gotchas

- **`field(default_factory=dict)` not `field(default={})`** — mutable default gotcha applies to dataclasses too
- **`@dataclass(frozen=True)` but containing mutable fields** — the dataclass is "frozen" but contained lists/dicts are still mutable
- **Pydantic V1 vs V2** — major breaking changes; `@validator` (V1) vs `@field_validator` (V2); check which version is installed

🔗 DevOps Connection

Alert event schemas, deployment request validation, Kubernetes event parsing, Terraform output parsing, webhook payload handling — all benefit from Pydantic's automatic validation and clear error messages.

---

23. Testing

📖 What It Is (Simple Terms)

`pytest` is the standard testing framework for Python SRE tools. Good tests for SRE scripts verify that they handle real-world edge cases — network failures, empty API responses, race conditions, malformed config files — not just the happy path.

🧩 Key Patterns

```python
# test_deploy.py
import pytest
from unittest.mock import patch, MagicMock, call
from pathlib import Path
import json

from myapp.deploy import deploy_service, DeploymentError

# ---- BASIC TESTS ----

def test_deploy_success():
    """Test successful deployment."""
```

```python
    # Arrange
    with patch("myapp.deploy.subprocess.run") as mock_run:
        mock_run.return_value = MagicMock(returncode=0,
stdout="deployment.apps/api-gateway image updated")

        # Act
        result = deploy_service("api-gateway", "v1.2.3", "production")

        # Assert
        assert result["status"] == "success"
        mock_run.assert_called_once()
        call_args = mock_run.call_args[0][0]
        assert "kubectl" in call_args
        assert "api-gateway" in call_args

def test_deploy_fails_on_kubectl_error():
    """Test that CalledProcessError is properly wrapped."""
    import subprocess
    with patch("myapp.deploy.subprocess.run") as mock_run:
        mock_run.side_effect = subprocess.CalledProcessError(
            returncode=1,
            cmd=["kubectl", "set", "image"],
            stderr="Error: deployment not found"
        )

        with pytest.raises(DeploymentError, match="deployment not found"):
            deploy_service("nonexistent", "v1.0.0", "production")

# ---- FIXTURES ----

@pytest.fixture
def mock_k8s_client():
    """Provide a mocked Kubernetes client."""
    with patch("myapp.k8s.CoreV1Api") as MockApi:
        client = MockApi.return_value
        client.list_namespaced_pod.return_value = MagicMock(
            items=[
                MagicMock(
                    metadata=MagicMock(name="api-abc123"),
                    status=MagicMock(phase="Running")
                )
            ]
        )
        yield client

@pytest.fixture
def sample_deployment_config(tmp_path: Path):
    """Create a sample config file in a temp directory."""
    config = {
        "service": "api-gateway",
        "namespace": "production",
        "replicas": 3,
    }
    config_file = tmp_path / "deploy_config.json"
```

```
    config_file.write_text(json.dumps(config))
    return config_file

def test_load_config(sample_deployment_config):
    from myapp.config import load_config
    config = load_config(str(sample_deployment_config))
    assert config["service"] == "api-gateway"
    assert config["replicas"] == 3

# ---- PARAMETRIZE ----

@pytest.mark.parametrize("exit_code,expected_status", [
    (0, "success"),
    (1, "general_error"),
    (137, "oomkilled"),
    (143, "sigterm"),
])
def test_exit_code_mapping(exit_code, expected_status):
    from myapp.utils import exit_code_to_status
    assert exit_code_to_status(exit_code) == expected_status

@pytest.mark.parametrize("version,valid", [
    ("v1.2.3", True),
    ("v1.2.3-rc1", True),
    ("1.2.3", False),      # missing v prefix
    ("v1.2", False),       # missing patch
    ("latest", False),     # not semver
    ("", False),           # empty
])
def test_version_validation(version, valid):
    from myapp.utils import is_valid_version
    assert is_valid_version(version) == valid

# ---- MOCKING HTTP ----

import responses  # pip install responses

@responses.activate
def test_prometheus_query():
    responses.add(
        responses.GET,
        "http://prometheus:9090/api/v1/query",
        json={
            "status": "success",
            "data": {
                "resultType": "vector",
                "result": [{"metric": {"job": "api"}, "value":
[1705315200, "0.05"]}]
            }
        },
        status=200,
    )

    from myapp.monitoring import PrometheusClient
```

```python
    client = PrometheusClient("http://prometheus:9090")
    results = client.query('up{job="api"}')

    assert len(results) == 1
    assert results[0]["metric"]["job"] == "api"

# ---- MOCKING TIME ----

from freezegun import freeze_time  # pip install freezegun

@freeze_time("2024-01-15 10:00:00")
def test_alert_timestamp():
    from myapp.alerts import create_alert
    alert = create_alert("test-service", "critical", "Service down")
    assert alert.timestamp.isoformat() == "2024-01-15T10:00:00"

# ---- ENVIRONMENT VARIABLES IN TESTS ----

@pytest.fixture(autouse=True)
def clean_env(monkeypatch):
    """Ensure clean environment for each test."""
    monkeypatch.delenv("AWS_PROFILE", raising=False)
    monkeypatch.setenv("AWS_REGION", "us-east-1")
    monkeypatch.setenv("ENVIRONMENT", "test")

def test_config_loads_from_env(monkeypatch):
    monkeypatch.setenv("DB_HOST", "test-db.internal")
    monkeypatch.setenv("DB_PORT", "5432")

    from myapp.config import DatabaseConfig
    config = DatabaseConfig.from_env()
    assert config.host == "test-db.internal"
    assert config.port == 5432

# ---- CONFTEST.PY (shared fixtures) ----
# conftest.py is auto-discovered by pytest
# pytest fixtures in conftest.py are available to all tests in the
directory

# ---- RUNNING TESTS ----
# pytest -v                          # verbose
# pytest -k "test_deploy"            # filter by name
# pytest --tb=short                  # shorter tracebacks
# pytest -x                         # stop on first failure
# pytest --cov=myapp --cov-report=html  # with coverage
```

💬 Short Interview Answer

"For SRE tool testing, the most important patterns are: use `unittest.mock.patch` to mock subprocess, HTTP calls, and external APIs so tests don't hit real infrastructure; use `pytest.fixture` with `tmp_path` for file-based tests; use `monkeypatch` for environment variable manipulation; use `@pytest.mark.parametrize` for testing multiple inputs. Test the unhappy paths — what happens when

kubectl exits non-zero, when the API returns 503, when the config file is malformed. Those are the scenarios that cause 3am incidents."

⚠ Tricky Gotchas

- **`patch` path must match where the function is used, not where it's defined** — `patch("myapp.deploy.subprocess.run")` not `patch("subprocess.run")`
- **`MagicMock.return_value`** is the value returned when the mock is called; **`MagicMock.side_effect`** raises the exception or calls a function
- **Fixtures with `yield` are teardown fixtures** — code after `yield` runs after the test
- **`tmp_path` is a built-in pytest fixture** — gives a unique temp directory per test, auto-cleaned

🔗 DevOps Connection

CI/CD pipelines run tests before deployments. SRE scripts that lack tests are deployment risks — untested edge cases become 3am incidents. Coverage reporting in CI ensures that new deployment script paths are always tested.

---

24. Package Structure

📖 What It Is (Simple Terms)

Organizing Python SRE tools as proper packages enables sharing across teams, version-pinning, and clean import structure. A well-structured package installs with `pip install` and can be versioned in a private PyPI registry.

🧩 Project Structure

```
my-sre-tools/
├── pyproject.toml           # modern Python packaging config
├── README.md
├── .gitignore
├── requirements.txt         # development dependencies
├── requirements-dev.txt     # test/lint dependencies
├── src/
│   └── sre_tools/
│       ├── __init__.py      # package version, main exports
│       ├── py.typed         # PEP 561: marks package as typed
│       ├── deploy/
│       │   ├── __init__.py
│       │   ├── kubernetes.py
│       │   └── canary.py
│       ├── monitoring/
│       │   ├── __init__.py
│       │   └── prometheus.py
│       ├── cli/
│       │   ├── __init__.py
│       │   └── main.py      # CLI entry point
│       └── utils/
│           ├── __init__.py
```

```
│           ├── retry.py
│           └── config.py
└── tests/
    ├── conftest.py
    ├── test_deploy/
    └── test_monitoring/
```

```toml
# pyproject.toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "sre-tools"
version = "1.2.3"
description = "Platform SRE automation toolkit"
requires-python = ">=3.11"
dependencies = [
    "requests>=2.31",
    "pyyaml>=6.0",
    "boto3>=1.28",
    "kubernetes>=28.0",
    "pydantic>=2.0",
    "pydantic-settings>=2.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.0",
    "pytest-cov",
    "responses",
    "freezegun",
    "mypy",
    "ruff",
]

[project.scripts]
sretool = "sre_tools.cli.main:main"  # creates `sretool` CLI command

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.mypy]
strict = true
python_version = "3.11"

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "--tb=short -q"
```

```python
# src/sre_tools/__init__.py
"""SRE Tools — Platform automation toolkit."""

__version__ = "1.2.3"
__author__ = "Platform Team"

# Re-export key public API
from .deploy.kubernetes import deploy_service, rollback_service
from .monitoring.prometheus import PrometheusClient
from .utils.retry import retry

__all__ = [
    "deploy_service",
    "rollback_service",
    "PrometheusClient",
    "retry",
]
```

🔗 DevOps Connection

Distributing SRE tools as pip packages via a private registry (AWS CodeArtifact, JFrog Artifactory, GitLab Package Registry) enables version-pinned deployments — `pip install sre-tools==1.2.3` — giving reproducible automation environments across teams.

🔴 EXPERT TOPICS

---

25. AWS SDK — boto3

📖 What It Is (Simple Terms)

`boto3` is the official AWS SDK for Python. It's how you automate AWS — EC2, S3, ECS, Lambda, Route53, CloudWatch, SSM, Secrets Manager, IAM — from Python scripts. It handles authentication, request signing, pagination, and retries automatically.

⚙️ How It Works Internally

boto3 is built on `botocore`. It generates client code from JSON service models. Two main interfaces: **client** (low-level, 1:1 with API) and **resource** (high-level, OOP). Authentication uses the credential chain: environment variables → `~/.aws/credentials` → IAM instance role → ECS task role.

```python
import boto3
from botocore.exceptions import ClientError, NoCredentialsError
from botocore.config import Config
import logging

logger = logging.getLogger(__name__)
```

```python
# ---- AUTHENTICATION ----

# Uses credential chain automatically (env vars, ~/.aws/credentials, IAM
role)
client = boto3.client("ec2", region_name="us-east-1")

# Assume role (cross-account, least privilege)
def get_assumed_role_client(role_arn: str, service: str, region: str =
"us-east-1"):
    sts = boto3.client("sts")
    response = sts.assume_role(
        RoleArn=role_arn,
        RoleSessionName="sre-automation",
        DurationSeconds=3600,
    )
    creds = response["Credentials"]
    return boto3.client(
        service,
        region_name=region,
        aws_access_key_id=creds["AccessKeyId"],
        aws_secret_access_key=creds["SecretAccessKey"],
        aws_session_token=creds["SessionToken"],
    )

prod_client = get_assumed_role_client(
    "arn:aws:iam::123456789012:role/SREAutomationRole",
    "ec2"
)

# ---- RETRY CONFIG ----

config = Config(
    retries={"max_attempts": 10, "mode": "adaptive"},
    max_pool_connections=50,
)
ec2 = boto3.client("ec2", config=config)

# ---- EC2 OPERATIONS ----

def get_instances_by_tag(tag_key: str, tag_value: str, region: str = "us-
east-1") -> list[dict]:
    """Get all EC2 instances with a specific tag."""
    ec2 = boto3.client("ec2", region_name=region)

    paginator = ec2.get_paginator("describe_instances")
    pages = paginator.paginate(
        Filters=[
            {"Name": f"tag:{tag_key}", "Values": [tag_value]},
            {"Name": "instance-state-name", "Values": ["running"]},
        ]
    )

    instances = []
```

```
    for page in pages:
        for reservation in page["Reservations"]:
            for instance in reservation["Instances"]:
                tags = {t["Key"]: t["Value"] for t in instance.get("Tags",
[])}
                instances.append({
                    "id": instance["InstanceId"],
                    "type": instance["InstanceType"],
                    "private_ip": instance.get("PrivateIpAddress"),
                    "public_ip": instance.get("PublicIpAddress"),
                    "az": instance["Placement"]["AvailabilityZone"],
                    "tags": tags,
                })

    return instances

# ---- S3 OPERATIONS ----

s3 = boto3.client("s3")

# Upload (large files: use multipart via upload_file, not put_object)
s3.upload_file(
    Filename="/tmp/deployment-artifact.tar.gz",
    Bucket="my-artifacts",
    Key="releases/api-gateway/v1.2.3/artifact.tar.gz",
    ExtraArgs={"ServerSideEncryption": "AES256"},
)

# Download
s3.download_file("my-artifacts", "releases/api/v1.2.3/artifact.tar.gz",
"/tmp/artifact.tar.gz")

# List objects with pagination
paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(Bucket="my-bucket", Prefix="releases/api-
gateway/"):
    for obj in page.get("Contents", []):
        print(obj["Key"], obj["LastModified"], obj["Size"])

# ---- SSM (PARAMETER STORE & SESSION MANAGER) ----

ssm = boto3.client("ssm", region_name="us-east-1")

# Get parameter
def get_parameter(name: str, decrypt: bool = True) -> str:
    response = ssm.get_parameter(Name=name, WithDecryption=decrypt)
    return response["Parameter"]["Value"]

db_password = get_parameter("/production/database/password")

# Get multiple parameters by path
def get_parameters_by_path(path: str) -> dict[str, str]:
    paginator = ssm.get_paginator("get_parameters_by_path")
    params = {}
```

```
    for page in paginator.paginate(Path=path, WithDecryption=True):
        for param in page["Parameters"]:
            key = param["Name"].split("/")[-1]
            params[key] = param["Value"]
    return params

config = get_parameters_by_path("/production/api-gateway/")

# Run command via SSM (no SSH needed!)
def run_command_on_instances(
    instance_ids: list[str],
    command: str,
    timeout_seconds: int = 60,
) -> dict:
    response = ssm.send_command(
        InstanceIds=instance_ids,
        DocumentName="AWS-RunShellScript",
        Parameters={"commands": [command]},
        TimeoutSeconds=timeout_seconds,
    )
    return response["Command"]

# ---- CLOUDWATCH METRICS ----

cloudwatch = boto3.client("cloudwatch", region_name="us-east-1")

def put_metric(namespace: str, metric_name: str, value: float, unit: str =
"None", **dimensions):
    cloudwatch.put_metric_data(
        Namespace=namespace,
        MetricData=[{
            "MetricName": metric_name,
            "Value": value,
            "Unit": unit,
            "Dimensions": [{"Name": k, "Value": v} for k, v in
dimensions.items()],
        }]
    )

put_metric("SRE/Deployments", "DeploymentDurationSeconds", 45.2,
"Seconds",
           Service="api-gateway", Environment="production")

# Get metric stats
response = cloudwatch.get_metric_statistics(
    Namespace="AWS/ApplicationELB",
    MetricName="TargetResponseTime",
    Dimensions=[{"Name": "LoadBalancer", "Value": "app/my-alb/abc123"}],
    StartTime=datetime.utcnow() - timedelta(minutes=60),
    EndTime=datetime.utcnow(),
    Period=300,
    Statistics=["Average", "p99"],
)
```

```
# ---- ECS OPERATIONS ----

ecs = boto3.client("ecs", region_name="us-east-1")

def deploy_ecs_service(cluster: str, service: str, image: str) -> None:
    """Update ECS service with new image via task definition revision."""

    # Get current task definition
    svc = ecs.describe_services(cluster=cluster, services=[service])
["services"][0]
    current_td_arn = svc["taskDefinition"]
    current_td =
ecs.describe_task_definition(taskDefinition=current_td_arn)
["taskDefinition"]

    # Update image in container definition
    containers = current_td["containerDefinitions"]
    for c in containers:
        if "api" in c["name"]:  # match by name
            c["image"] = image

    # Register new task definition
    new_td = ecs.register_task_definition(
        family=current_td["family"],
        containerDefinitions=containers,
        executionRoleArn=current_td["executionRoleArn"],
        taskRoleArn=current_td.get("taskRoleArn", ""),
        networkMode=current_td["networkMode"],
        requiresCompatibilities=current_td["requiresCompatibilities"],
        cpu=current_td.get("cpu"),
        memory=current_td.get("memory"),
    )
    new_td_arn = new_td["taskDefinition"]["taskDefinitionArn"]

    # Update service
    ecs.update_service(
        cluster=cluster,
        service=service,
        taskDefinition=new_td_arn,
        forceNewDeployment=True,
    )
    logger.info(f"Updated {service} to {new_td_arn}")

# ---- ERROR HANDLING ----

try:
    ec2.describe_instances()
except ClientError as e:
    error_code = e.response["Error"]["Code"]
    error_msg = e.response["Error"]["Message"]

    if error_code == "UnauthorizedOperation":
        raise PermissionError(f"Insufficient AWS permissions:
{error_msg}")
```

```
    elif error_code == "RequestLimitExceeded":
        logger.warning("AWS API rate limit hit, backing off...")
        time.sleep(5)
    else:
        raise
except NoCredentialsError:
    raise RuntimeError("AWS credentials not configured")
```

💬 Short Interview Answer

"boto3 has two main interfaces — client (low-level, maps 1:1 to AWS API operations) and resource (high-level OOP). For production scripts, always use paginators for list operations — `describe_instances`, `list_objects_v2` etc. only return up to 1000 results per call. Handle `ClientError` specifically — check `e.response['Error']['Code']` to distinguish 'resource not found' from 'permission denied'. For cross-account access, use STS assume_role and create a client with the temporary credentials. Always configure retry mode — use 'adaptive' for CLI tools and 'standard' for high-throughput services."

⚠ Tricky Gotchas

- **Missing paginator = missing data** — `describe_instances` returns max 1000 results; you MUST paginate
- **boto3 sessions are NOT thread-safe** — create separate clients per thread in concurrent code
- **`aws_session_token` is required** when using assumed role credentials
- **ClientError code "404" varies by service** — S3 uses `NoSuchKey`, EC2 uses `InvalidInstanceID.NotFound`, etc.

🔗 DevOps Connection

EC2 instance management, S3 artifact storage, ECS deployments, CloudWatch custom metrics, SSM parameter store for config/secrets, Auto Scaling group management, Route53 health check management — all via boto3 in SRE automation.

---

26. GCP SDK

📖 What It Is (Simple Terms)

Google Cloud's Python SDK (`google-cloud-*` libraries) lets you automate GCP resources — GKE clusters, Cloud Storage, Cloud SQL, Pub/Sub, Cloud Run, Monitoring, Secret Manager — from Python.

```
# ---- AUTHENTICATION ----
# pip install google-auth google-cloud-storage google-cloud-container

from google.oauth2 import service_account
from google.auth import default as google_auth_default
import google.auth.transport.requests

# Application Default Credentials (ADC) — preferred
credentials, project = google_auth_default()
```

```python
# Service account from file
credentials = service_account.Credentials.from_service_account_file(
    "service-account.json",
    scopes=["https://www.googleapis.com/auth/cloud-platform"],
)

# ---- CLOUD STORAGE ----

from google.cloud import storage

gcs = storage.Client()

# Upload
bucket = gcs.bucket("my-artifacts")
blob = bucket.blob("releases/api/v1.2.3/artifact.tar.gz")
blob.upload_from_filename("/tmp/artifact.tar.gz")

# Download
blob.download_to_filename("/tmp/artifact.tar.gz")

# List objects
for blob in gcs.list_blobs("my-artifacts", prefix="releases/api/"):
    print(blob.name, blob.size, blob.updated)

# ---- CONTAINER REGISTRY / ARTIFACT REGISTRY ----

from google.cloud.artifactregistry_v1 import ArtifactRegistryClient

client = ArtifactRegistryClient()
parent = "projects/my-project/locations/us-central1/repositories/my-repo"
packages = client.list_packages(parent=parent)

# ---- GKE CLUSTER MANAGEMENT ----

from google.cloud import container_v1

gke = container_v1.ClusterManagerClient()

# Get cluster info
cluster = gke.get_cluster(
    name="projects/my-project/locations/us-central1/clusters/prod-cluster"
)
print(f"Cluster: {cluster.name}, Status: {cluster.status.name}")

# ---- CLOUD MONITORING ----

from google.cloud import monitoring_v3
from google.protobuf import timestamp_pb2
import time

monitoring = monitoring_v3.MetricServiceClient()

def write_custom_metric(project: str, metric_type: str, value: float) ->
```

```
None:
    series = monitoring_v3.TimeSeries()
    series.metric.type = f"custom.googleapis.com/{metric_type}"

    now = time.time()
    point = monitoring_v3.Point()
    point.value.double_value = value
    point.interval.end_time.seconds = int(now)
    series.points = [point]

    monitoring.create_time_series(
        name=f"projects/{project}",
        time_series=[series]
    )

# ---- SECRET MANAGER ----

from google.cloud import secretmanager

sm = secretmanager.SecretManagerServiceClient()

def get_secret(project: str, secret_id: str, version: str = "latest") ->
str:
    name = f"projects/{project}/secrets/{secret_id}/versions/{version}"
    response = sm.access_secret_version(request={"name": name})
    return response.payload.data.decode("UTF-8")

db_password = get_secret("my-project", "db-password")
```

💬 Short Interview Answer

"The GCP SDK uses Application Default Credentials automatically — if you're running on a GCE instance, GKE pod with Workload Identity, or have `gcloud auth application-default login` configured, no explicit credential config is needed. The pattern across all GCP services is consistent — create a client, then call methods on it. For Pub/Sub event-driven automation, GCS for artifact storage, Secret Manager for secrets — these are the three GCP services I use most in SRE automation."

---

27. Kubernetes Automation

📖 What It Is (Simple Terms)

The official `kubernetes` Python client lets you interact with the Kubernetes API to list pods, watch events, manage deployments, create resources, and build controllers — directly from Python, without shelling out to kubectl.

```
# pip install kubernetes
from kubernetes import client, config, watch
from kubernetes.client.exceptions import ApiException
import yaml
```

```python
import logging

logger = logging.getLogger(__name__)

# ---- AUTHENTICATION ----

# Load from ~/.kube/config (local dev)
config.load_kube_config()

# Load from in-cluster service account (when running in a pod)
config.load_incluster_config()

# Auto-detect: try in-cluster first, fall back to kubeconfig
try:
    config.load_incluster_config()
except config.ConfigException:
    config.load_kube_config()

# ---- CLIENTS ----

v1 = client.CoreV1Api()           # pods, services, configmaps, secrets,
nodes
apps_v1 = client.AppsV1Api()     # deployments, statefulsets, daemonsets
batch_v1 = client.BatchV1Api()   # jobs, cronjobs
rbac_v1 = client.RbacAuthorizationV1Api()  # roles, clusterroles

# ---- POD OPERATIONS ----

def get_running_pods(namespace: str) -> list[dict]:
    """List all running pods in a namespace."""
    pods = v1.list_namespaced_pod(
        namespace=namespace,
        field_selector="status.phase=Running"
    )
    return [
        {
            "name": pod.metadata.name,
            "namespace": pod.metadata.namespace,
            "node": pod.spec.node_name,
            "ip": pod.status.pod_ip,
            "containers": [c.name for c in pod.spec.containers],
            "labels": pod.metadata.labels or {},
            "phase": pod.status.phase,
        }
        for pod in pods.items
    ]

def get_pod_logs(namespace: str, pod_name: str, container: str = None,
tail: int = 100) -> str:
    return v1.read_namespaced_pod_log(
        name=pod_name,
        namespace=namespace,
        container=container,
        tail_lines=tail,
```

```
        timestamps=True,
    )

def delete_crashlooping_pods(namespace: str) -> list[str]:
    """Delete pods in CrashLoopBackOff state."""
    pods = v1.list_namespaced_pod(namespace=namespace)
    deleted = []

    for pod in pods.items:
        container_statuses = pod.status.container_statuses or []
        for status in container_statuses:
            if (status.state.waiting and
                status.state.waiting.reason == "CrashLoopBackOff"):

                v1.delete_namespaced_pod(
                    name=pod.metadata.name,
                    namespace=namespace,
                )
                deleted.append(pod.metadata.name)
                logger.info(f"Deleted CrashLoopBackOff pod:
{pod.metadata.name}")
                break

    return deleted

# ---- DEPLOYMENT OPERATIONS ----

def scale_deployment(name: str, namespace: str, replicas: int) -> None:
    """Scale a deployment to a specific replica count."""
    apps_v1.patch_namespaced_deployment_scale(
        name=name,
        namespace=namespace,
        body={"spec": {"replicas": replicas}},
    )
    logger.info(f"Scaled {namespace}/{name} to {replicas} replicas")

def get_deployment_status(name: str, namespace: str) -> dict:
    deployment = apps_v1.read_namespaced_deployment(name=name,
namespace=namespace)
    return {
        "name": name,
        "desired": deployment.spec.replicas,
        "ready": deployment.status.ready_replicas or 0,
        "available": deployment.status.available_replicas or 0,
        "updated": deployment.status.updated_replicas or 0,
        "conditions": [
            {"type": c.type, "status": c.status, "reason": c.reason}
            for c in (deployment.status.conditions or [])
        ],
    }

def wait_for_rollout(name: str, namespace: str, timeout: int = 300) ->
bool:
    """Wait for deployment rollout to complete."""
```

```
    import time
    deadline = time.time() + timeout

    while time.time() < deadline:
        status = get_deployment_status(name, namespace)
        if status["ready"] == status["desired"] and status["updated"] ==
status["desired"]:
            logger.info(f"Rollout complete: {name}
({status['ready']}/{status['desired']} ready)")
            return True

        logger.info(f"Waiting: {name}
{status['ready']}/{status['desired']} ready...")
        time.sleep(10)

    logger.error(f"Rollout timed out after {timeout}s: {status}")
    return False

# ---- WATCH API (streaming events) ----

def watch_pods(namespace: str, label_selector: str = None) -> None:
    """Watch pod events in real time."""
    w = watch.Watch()
    for event in w.stream(
        v1.list_namespaced_pod,
        namespace=namespace,
        label_selector=label_selector,
        timeout_seconds=300,
    ):
        event_type = event["type"]   # ADDED, MODIFIED, DELETED
        pod = event["object"]
        pod_name = pod.metadata.name
        phase = pod.status.phase

        if event_type == "MODIFIED" and phase == "Failed":
            logger.error(f"Pod failed: {pod_name}")
            page_oncall(f"Pod {pod_name} in {namespace} failed!")

# ---- APPLY MANIFEST FROM YAML ----

def apply_manifest(manifest_yaml: str, namespace: str = "default") ->
None:
    """Apply a Kubernetes manifest (like kubectl apply)."""
    from kubernetes.utils import create_from_yaml
    import tempfile, os

    with tempfile.NamedTemporaryFile(mode='w', suffix='.yaml',
delete=False) as f:
        f.write(manifest_yaml)
        tmp_path = f.name

    try:
        k8s_client = client.ApiClient()
        create_from_yaml(k8s_client, tmp_path, namespace=namespace)
```

```
    finally:
        os.unlink(tmp_path)

# ---- ERROR HANDLING ----

try:
    deployment = apps_v1.read_namespaced_deployment("my-app",
"production")
except ApiException as e:
    if e.status == 404:
        print("Deployment not found")
    elif e.status == 403:
        print("Forbidden — check RBAC permissions")
    else:
        raise
```

💬 Short Interview Answer

"The kubernetes Python client mirrors the kubectl API — you have `CoreV1Api` for pods/services/configmaps, `AppsV1Api` for deployments, and so on. The Watch API is powerful — it streams events from the API server using long-polling, letting you build event-driven automation like automatically restarting CrashLoopBackOff pods or triggering alerts when deployments fail. For scripts running inside a cluster, use `load_incluster_config()` which reads the service account token automatically mounted at `/var/run/secrets/kubernetes.io/serviceaccount/`."

⚠ Tricky Gotchas

- **`status.ready_replicas` can be `None`** (not 0) when no pods are ready — always use `or 0`
- **The Watch stream can disconnect** — wrap in a retry loop for production watchers
- **`create_from_yaml` doesn't support `apply` semantics** — it creates, not patches existing resources
- **API rate limiting** — Kubernetes API server has rate limits; add exponential backoff for automated scripts

🔗 DevOps Connection

Automated deployment verification, pod health monitoring, auto-remediation (restart crashlooping pods), GitOps controllers, canary deployment automation, cluster capacity management — all built on the Kubernetes Python client.

---

28. Docker Automation

📖 What It Is (Simple Terms)

The `docker` Python SDK lets you build images, manage containers, interact with registries, and inspect Docker state — from Python scripts, without shelling out to docker CLI.

```python
# pip install docker
import docker
import tarfile
import io

client = docker.from_env()  # connects to Docker daemon via socket

# ---- IMAGE OPERATIONS ----

# Build image
image, build_logs = client.images.build(
    path=".",           # build context directory
    tag="myapp:v1.2.3",
    dockerfile="Dockerfile",
    rm=True,            # remove intermediate containers
    buildargs={"VERSION": "v1.2.3"},
)
for log in build_logs:
    if "stream" in log:
        print(log["stream"].rstrip())

# Pull image
image = client.images.pull("redis:7-alpine")

# Push image
for line in client.images.push("myregistry.io/myapp:v1.2.3", stream=True,
decode=True):
    if "status" in line:
        print(line["status"])

# Tag image
image.tag("myregistry.io/myapp:latest")

# ---- CONTAINER OPERATIONS ----

# Run a container (like docker run)
container = client.containers.run(
    "postgres:15",
    name="test-postgres",
    environment={
        "POSTGRES_PASSWORD": "test",
        "POSTGRES_DB": "testdb",
    },
    ports={"5432/tcp": 5432},
    volumes={"/tmp/pgdata": {"bind": "/var/lib/postgresql/data", "mode":
"rw"}},
    detach=True,
    remove=False,
)

# Wait for container
result = container.wait()
exit_code = result["StatusCode"]
```

```
# Get logs
logs = container.logs(stream=False, follow=False).decode()

# Execute command in running container
exec_result = container.exec_run(
    "pg_isready -U postgres",
    user="postgres",
)
print(exec_result.output.decode())

# Container cleanup
container.stop(timeout=10)
container.remove(force=True)

# ---- HEALTH CHECK WAITING ----

import time

def wait_for_healthy(container, timeout: int = 60) -> bool:
    deadline = time.time() + timeout
    while time.time() < deadline:
        container.reload()
        health = container.attrs.get("State", {}).get("Health", {})
        status = health.get("Status", "none")
        if status == "healthy":
            return True
        elif status == "unhealthy":
            return False
        time.sleep(2)
    return False

# ---- IMAGE SCANNING ----

def get_image_layers(image_name: str) -> list[dict]:
    image = client.images.get(image_name)
    return image.history()

def check_image_size(image_name: str, max_mb: float = 500) -> bool:
    image = client.images.get(image_name)
    size_mb = image.attrs["Size"] / (1024 * 1024)
    if size_mb > max_mb:
        print(f"WARNING: Image {image_name} is {size_mb:.1f}MB (max:
{max_mb}MB)")
        return False
    return True
```

---

29. Prometheus and Monitoring

📖 What It Is (Simple Terms)

The `prometheus_client` library lets Python scripts expose metrics (counters, gauges, histograms) that Prometheus can scrape. For SRE automation scripts, this means you can track how often your deploy script runs, how long it takes, and how often it fails — all in Prometheus/Grafana.

```
# pip install prometheus-client
from prometheus_client import (
    Counter, Gauge, Histogram, Summary, push_to_gateway,
    start_http_server, CollectorRegistry, REGISTRY
)
import time
import functools

# ---- METRIC TYPES ----

# Counter — monotonically increasing (events, errors)
deployment_total = Counter(
    "sre_deployments_total",
    "Total number of deployments",
    ["service", "environment", "status"]  # labels
)

# Gauge — can go up and down (queue depth, current replicas, temperature)
pod_count = Gauge(
    "sre_pod_count",
    "Current number of running pods",
    ["service", "namespace"]
)

# Histogram — distribution of values (request duration, payload size)
deployment_duration = Histogram(
    "sre_deployment_duration_seconds",
    "Time taken for deployments to complete",
    ["service", "environment"],
    buckets=[30, 60, 120, 300, 600, float("inf")]
)

# ---- RECORDING METRICS ----

deployment_total.labels(service="api-gateway", environment="prod",
status="success").inc()
pod_count.labels(service="api-gateway", namespace="production").set(5)

with deployment_duration.labels(service="api-gateway",
environment="prod").time():
    run_deployment()  # automatically records duration

# ---- DECORATOR FOR TRACKING ----

def track_deployment(service: str, environment: str):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
```

```
            start = time.time()
            try:
                result = func(*args, **kwargs)
                deployment_total.labels(
                    service=service, environment=environment,
status="success"
                ).inc()
                return result
            except Exception as e:
                deployment_total.labels(
                    service=service, environment=environment,
status="failure"
                ).inc()
                raise
            finally:
                deployment_duration.labels(
                    service=service, environment=environment
                ).observe(time.time() - start)
        return wrapper
    return decorator

@track_deployment("api-gateway", "production")
def deploy_api_gateway():
    pass

# ---- PUSHGATEWAY (for batch jobs) ----

registry = CollectorRegistry()
g = Gauge("backup_last_success_timestamp", "Last backup success time",
registry=registry)
g.set(time.time())

push_to_gateway(
    "prometheus-pushgateway:9091",
    job="sre-backup-job",
    registry=registry,
    grouping_key={"environment": "production"}
)

# ---- HTTP METRICS ENDPOINT ----

# Expose metrics on :8000/metrics for Prometheus scraping
start_http_server(8000)

# ---- QUERYING PROMETHEUS ----

import requests

class PrometheusClient:
    def __init__(self, url: str):
        self.url = url
        self.session = requests.Session()

    def query(self, promql: str) -> list:
```

```
        resp = self.session.get(
            f"{self.url}/api/v1/query",
            params={"query": promql},
            timeout=30,
        )
        resp.raise_for_status()
        data = resp.json()
        if data["status"] != "success":
            raise RuntimeError(f"Query failed: {data.get('error')}")
        return data["data"]["result"]

    def get_error_rate(self, service: str, window: str = "5m") -> float:
        results = self.query(
            f'sum(rate(http_requests_total{{service="
{service}",code=~"5.."}}[{window}])) / '
            f'sum(rate(http_requests_total{{service="{service}"}}
[{window}]))'
        )
        return float(results[0]["value"][1]) if results else 0.0

    def is_slo_breached(self, service: str, slo_target: float = 0.999) ->
bool:
        error_rate = self.get_error_rate(service)
        availability = 1 - error_rate
        return availability < slo_target
```

---

30. CI/CD Scripting Patterns

📖 What It Is (Simple Terms)

Python scripts used in CI/CD pipelines (GitHub Actions, GitLab CI, Jenkins, CircleCI) perform tasks like running tests, building images, deploying to environments, sending notifications, validating configs, and implementing custom gates.

🧩 Key Patterns

```
#!/usr/bin/env python3
"""
ci_gate.py — Custom CI gate: validates PRs before merging to main
"""
import os
import sys
import json
import subprocess
import requests
import logging
from pathlib import Path

logging.basicConfig(level=logging.INFO, format="%(levelname)s %
(message)s")
```

```
logger = logging.getLogger(__name__)

# ---- GITHUB API INTEGRATION ----

GITHUB_TOKEN = os.environ.get("GITHUB_TOKEN")
GITHUB_API = "https://api.github.com"

def github_request(method: str, path: str, **kwargs) -> dict:
    resp = requests.request(
        method,
        f"{GITHUB_API}{path}",
        headers={
            "Authorization": f"Bearer {GITHUB_TOKEN}",
            "Accept": "application/vnd.github.v3+json",
        },
        timeout=30,
        **kwargs,
    )
    resp.raise_for_status()
    return resp.json()

def get_pr_files(repo: str, pr_number: int) -> list[str]:
    files = github_request("GET",
f"/repos/{repo}/pulls/{pr_number}/files")
    return [f["filename"] for f in files]

def set_commit_status(repo: str, sha: str, state: str, description: str,
context: str) -> None:
    github_request("POST", f"/repos/{repo}/statuses/{sha}", json={
        "state": state,         # "pending", "success", "failure", "error"
        "description": description[:140],
        "context": context,     # "ci/deploy-gate"
    })

# ---- CI ENVIRONMENT DETECTION ----

def is_pull_request() -> bool:
    return bool(
        os.environ.get("GITHUB_EVENT_NAME") == "pull_request" or
        os.environ.get("CI_MERGE_REQUEST_ID") or  # GitLab
        os.environ.get("CHANGE_ID")               # Jenkins multibranch
    )

def get_current_branch() -> str:
    # Try CI env vars first
    for var in ["GITHUB_HEAD_REF", "CI_COMMIT_BRANCH", "GIT_BRANCH",
"BRANCH_NAME"]:
        if os.environ.get(var):
            return os.environ[var]
    # Fall back to git
    result = subprocess.run(
        ["git", "rev-parse", "--abbrev-ref", "HEAD"],
        capture_output=True, text=True, check=True
    )
```

```
    return result.stdout.strip()

# ---- DEPLOYMENT GATE CHECKS ----

def check_test_coverage(min_coverage: float = 80.0) -> tuple[bool, str]:
    """Run pytest with coverage and check threshold."""
    result = subprocess.run(
        ["pytest", "--cov=src", "--cov-report=json", "-q"],
        capture_output=True, text=True
    )
    if result.returncode != 0:
        return False, "Tests failed"

    try:
        with open(".coverage.json") as f:
            coverage_data = json.load(f)
        total = coverage_data["totals"]["percent_covered"]
        if total < min_coverage:
            return False, f"Coverage {total:.1f}% < threshold
{min_coverage}%"
        return True, f"Coverage {total:.1f}% ✓"
    except (FileNotFoundError, KeyError):
        return False, "Coverage report not found"

def check_security_scan() -> tuple[bool, str]:
    """Run Bandit security scanner."""
    result = subprocess.run(
        ["bandit", "-r", "src/", "-f", "json", "-ll"],  # high severity
only
        capture_output=True, text=True
    )
    if result.returncode == 0:
        return True, "No high-severity security issues"

    try:
        findings = json.loads(result.stdout)
        count = len(findings.get("results", []))
        return False, f"{count} high-severity security issue(s) found"
    except json.JSONDecodeError:
        return False, "Security scan failed to parse results"

def check_docker_build(dockerfile: str = "Dockerfile") -> tuple[bool,
str]:
    """Verify Docker image builds successfully."""
    result = subprocess.run(
        ["docker", "build", "--no-cache", "-f", dockerfile, "-t", "ci-
test:latest", "."],
        capture_output=True, text=True, timeout=300
    )
    if result.returncode == 0:
        return True, "Docker build successful"
    return False, f"Docker build failed: {result.stderr[-500:]}"

# ---- MAIN CI GATE RUNNER ----
```

```python
def run_ci_gates() -> int:
    gates = [
        ("Test Coverage", lambda: check_test_coverage(80.0)),
        ("Security Scan", check_security_scan),
        ("Docker Build", check_docker_build),
    ]

    failures = []
    for name, check in gates:
        logger.info(f"Running gate: {name}")
        try:
            passed, message = check()
            if passed:
                logger.info(f"  ✓ {name}: {message}")
            else:
                logger.error(f"  ✗ {name}: {message}")
                failures.append((name, message))
        except Exception as e:
            logger.error(f"  ✗ {name}: Exception: {e}")
            failures.append((name, str(e)))

    if failures:
        logger.error(f"\n{len(failures)} gate(s) failed:")
        for name, msg in failures:
            logger.error(f"  - {name}: {msg}")
        return 1

    logger.info("\nAll CI gates passed! ✓")
    return 0

if __name__ == "__main__":
    sys.exit(run_ci_gates())
```

GitHub Actions Integration

```yaml
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v4
```

```
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run CI gates
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          AWS_REGION: us-east-1
        run: python ci_gate.py

      - name: Deploy to production
        if: success()
        env:
          KUBECONFIG_DATA: ${{ secrets.KUBECONFIG_PRODUCTION }}
        run: |
          echo "$KUBECONFIG_DATA" | base64 -d > /tmp/kubeconfig
          KUBECONFIG=/tmp/kubeconfig python deploy.py \
            --service api-gateway \
            --version ${{ github.sha }} \
            --namespace production
```

---

31. Infrastructure as Code with Python

📖 What It Is (Simple Terms)

Pulumi and AWS CDK let you define infrastructure using Python instead of HCL (Terraform) or YAML (CloudFormation). You get loops, functions, classes, and IDE support for infrastructure code.

```python
# ---- PULUMI (pip install pulumi pulumi-aws) ----
import pulumi
import pulumi_aws as aws

# Create VPC
vpc = aws.ec2.Vpc(
    "prod-vpc",
    cidr_block="10.0.0.0/16",
    enable_dns_hostnames=True,
    tags={"Name": "prod-vpc", "Environment": "production"},
)

# Create subnets in each AZ
azs = ["us-east-1a", "us-east-1b", "us-east-1c"]
subnets = [
    aws.ec2.Subnet(
        f"private-subnet-{i}",
        vpc_id=vpc.id,
        cidr_block=f"10.0.{i}.0/24",
        availability_zone=az,
```

```
        tags={"Name": f"private-{az}"},
    )
    for i, az in enumerate(azs)
]

# EKS Cluster
eks_cluster = aws.eks.Cluster(
    "prod-cluster",
    role_arn=eks_role.arn,
    vpc_config=aws.eks.ClusterVpcConfigArgs(
        subnet_ids=[s.id for s in subnets],
    ),
    version="1.28",
)

# Export outputs
pulumi.export("cluster_endpoint", eks_cluster.endpoint)
pulumi.export("kubeconfig", eks_cluster.name)

# ---- AWS CDK (pip install aws-cdk-lib) ----
import aws_cdk as cdk
from aws_cdk import aws_ec2 as ec2, aws_ecs as ecs, aws_ecs_patterns as
ecs_patterns

class ApiServiceStack(cdk.Stack):
    def __init__(self, scope: cdk.App, id: str, **kwargs):
        super().__init__(scope, id, **kwargs)

        vpc = ec2.Vpc(self, "VPC", max_azs=3)
        cluster = ecs.Cluster(self, "Cluster", vpc=vpc)

        # Fargate service with load balancer
        service = ecs_patterns.ApplicationLoadBalancedFargateService(
            self, "ApiService",
            cluster=cluster,
            cpu=512,
            memory_limit_mib=1024,
            desired_count=3,

task_image_options=ecs_patterns.ApplicationLoadBalancedTaskImageOptions(

image=ecs.ContainerImage.from_registry("myregistry/api:v1.2.3"),
                container_port=8080,
            ),
            public_load_balancer=True,
        )

app = cdk.App()
ApiServiceStack(app, "ApiService-Production",
env=cdk.Environment(region="us-east-1"))
app.synth()
```

32. Secret Management

📖 What It Is (Simple Terms)

Production secrets (database passwords, API keys, TLS certificates) should never be in code or environment files. Python scripts should fetch them at runtime from Vault (HashiCorp), AWS Secrets Manager, GCP Secret Manager, or Azure Key Vault.

```
import os
import json
import boto3
import logging
from functools import lru_cache

logger = logging.getLogger(__name__)

# ---- AWS SECRETS MANAGER ----

@lru_cache(maxsize=None)
def get_aws_secret(secret_name: str, region: str = "us-east-1") -> dict |
str:
    """Fetch and cache secret from AWS Secrets Manager."""
    client = boto3.client("secretsmanager", region_name=region)
    try:
        response = client.get_secret_value(SecretId=secret_name)
    except client.exceptions.NoSuchResourceException:
        raise KeyError(f"Secret not found: {secret_name}")

    secret = response.get("SecretString") or response.get("SecretBinary",
b"").decode()
    try:
        return json.loads(secret)  # return dict if JSON
    except json.JSONDecodeError:
        return secret  # return string if not JSON

# Usage
db_creds = get_aws_secret("production/postgresql/credentials")
conn_str = f"postgresql://{db_creds['username']}:
{db_creds['password']}@{db_creds['host']}"

# ---- HASHICORP VAULT ----

import hvac  # pip install hvac

def get_vault_client(vault_addr: str, token: str | None = None) ->
hvac.Client:
    client = hvac.Client(url=vault_addr)

    if token:
        client.token = token
    elif os.environ.get("VAULT_TOKEN"):
        client.token = os.environ["VAULT_TOKEN"]
```

```
    else:
        # Kubernetes auth (if running in a pod)
        with open("/var/run/secrets/kubernetes.io/serviceaccount/token")
as f:
            jwt_token = f.read()
        client.auth.kubernetes.login(role="sre-tools", jwt=jwt_token)

    if not client.is_authenticated():
        raise RuntimeError("Vault authentication failed")

    return client

def get_vault_secret(path: str, key: str) -> str:
    """Read a secret from Vault KV v2."""
    vault = get_vault_client(os.environ["VAULT_ADDR"])
    response = vault.secrets.kv.v2.read_secret_version(
        path=path,
        mount_point="secret",
    )
    return response["data"]["data"][key]

db_password = get_vault_secret("production/database", "password")

# ---- NEVER DO THESE ----
# ❌ Hardcoded secrets
DB_PASSWORD = "super_secret_password_123"

# ❌ Secrets in environment files committed to git
# DB_PASSWORD=super_secret_password_123  in .env (if .env is tracked)

# ❌ Secrets in logs
logger.info(f"Connecting with password: {db_password}")  # BAD

# ✅ Always redact in logs
logger.info(f"Connecting to database at {db_host}")
```

---

33. Database Automation

📖 What It Is (Simple Terms)

SRE scripts often need to query databases directly — for health checks, data migrations, runbook automation, and incident response. `psycopg2` (PostgreSQL), `pymysql` (MySQL), and `SQLAlchemy` (ORM for multiple databases) are the key libraries.

```
import psycopg2
import psycopg2.extras
from contextlib import contextmanager
from typing import Iterator

# ---- CONNECTION MANAGEMENT ----
```

```
@contextmanager
def get_db_connection(dsn: str) ->
Iterator[psycopg2.extensions.connection]:
    """Context manager for database connection with auto-rollback on
error."""
    conn = psycopg2.connect(dsn)
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

# ---- HEALTH CHECKS ----

def check_postgres_health(dsn: str) -> dict:
    try:
        with get_db_connection(dsn) as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")

                # Get key metrics
                cur.execute("""
                    SELECT
                        pg_database_size(current_database()) as db_size,
                        (SELECT count(*) FROM pg_stat_activity WHERE state
= 'active') as active_conns,
                        (SELECT count(*) FROM pg_stat_activity WHERE
wait_event_type = 'Lock') as waiting_conns
                """)
                row = cur.fetchone()

                return {
                    "healthy": True,
                    "db_size_bytes": row[0],
                    "active_connections": row[1],
                    "waiting_on_lock": row[2],
                }
    except psycopg2.OperationalError as e:
        return {"healthy": False, "error": str(e)}

# ---- DICT CURSOR (returns dict rows) ----

def get_slow_queries(dsn: str, min_duration_ms: float = 1000) ->
list[dict]:
    with get_db_connection(dsn) as conn:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as
cur:
            cur.execute("""
                SELECT
                    query,
```

```
                    calls,
                    total_exec_time / calls as avg_ms,
                    max_exec_time as max_ms
                FROM pg_stat_statements
                WHERE total_exec_time / calls > %s
                ORDER BY avg_ms DESC
                LIMIT 20
            """, (min_duration_ms,))
            return [dict(row) for row in cur.fetchall()]

# ---- EXECUTE MANY (bulk operations) ----

def bulk_insert_events(dsn: str, events: list[dict]) -> int:
    with get_db_connection(dsn) as conn:
        with conn.cursor() as cur:
            psycopg2.extras.execute_values(
                cur,
                "INSERT INTO audit_events (service, event, timestamp)
VALUES %s",
                [(e["service"], e["event"], e["timestamp"]) for e in
events],
                page_size=1000,
            )
            return cur.rowcount
```

---

34. gRPC and Advanced APIs

📖 What It Is (Simple Terms)

gRPC is a high-performance RPC framework used by Kubernetes, etcd, and many microservices. Python's `grpc` library lets you call gRPC services. For Kubernetes, the Python client uses the REST API, but etcd and some service meshes use gRPC directly.

```
# pip install grpcio grpcio-tools

# Example: calling a gRPC health check service
import grpc
from grpc_health.v1 import health_pb2, health_pb2_grpc

def check_grpc_health(host: str, port: int, service: str = "") -> str:
    with grpc.insecure_channel(f"{host}:{port}") as channel:
        stub = health_pb2_grpc.HealthStub(channel)
        try:
            response = stub.Check(
                health_pb2.HealthCheckRequest(service=service),
                timeout=5.0,
            )
            return
health_pb2.HealthCheckResponse.ServingStatus.Name(response.status)
        except grpc.RpcError as e:
```

```
            return f"ERROR: {e.code()}: {e.details()}"

status = check_grpc_health("my-service.internal", 50051,
"my.service.Name")
print(status)  # "SERVING"

# ---- STREAMING RPC ----

def stream_log_events(stub, service: str):
    """Consume a server-streaming RPC."""
    request = LogRequest(service=service)
    for event in stub.StreamLogs(request):
        yield event.message, event.timestamp
```

🎯 INTERVIEW QUICK REFERENCE

Most Frequently Asked Questions — Short Answers Ready

Question	Key Answer
GIL and threading	GIL allows only one thread to execute Python bytecode at a time; I/O releases GIL so threading works for I/O-bound; use multiprocessing for CPU-bound
Memory leak in Python	Reference cycles (circular references), unclosed file handles, class-level mutable defaults accumulating state, global caches without eviction
`deepcopy` vs `copy`	`copy.copy()` is shallow (copies object, references inner objects); `deepcopy()` recursively copies everything
How would you script a rolling restart?	kubectl rollout restart, watch pod events with Watch API, verify health after each batch, auto-rollback on failure
Detect drift between desired and actual	List desired config, list actual (kubectl/boto3), set difference, alert on additions/removals/changes
How to make a script idempotent	Check-then-act pattern: check if already in desired state before making changes; use `exist_ok=True`, `--if-exists`, upserts
Fastest way to process 10M log lines	Generator pipeline (not loading all into memory), multiprocessing for CPU-bound parsing, split file into chunks
Why asyncio for many health checks?	Single thread, no lock overhead, handles hundreds of concurrent I/O ops; threads have ~8MB stack overhead each
How do you handle secrets in scripts?	Fetch from Secrets Manager/Vault at startup, never log them, use short-lived credentials (STS assume_role, Vault leases)

Question	Key Answer
What is a context manager used for?	Guaranteed resource cleanup — file handles, DB connections, distributed locks, temp files, AWS session tokens

⚠ Top 10 Most Common Interview Gotchas

1. **Mutable default argument** — `def f(x=[])` shares the list across calls
2. **`is` vs `==`** — `is` checks identity, `==` checks value; only use `is` for None/True/False
3. **`yaml.load()` is dangerous** — always `yaml.safe_load()`
4. **No timeout in requests** — `requests.get(url)` can hang forever
5. **Missing `@wraps` in decorators** — loses `__name__`, `__doc__`
6. **Subprocess shell injection** — always use list form, not `shell=True` with user input
7. **Modifying list while iterating** — undefined behavior; use comprehension
8. **`assert` is disabled with `-O`** — never use for production validation
9. **boto3 without paginator** — max 1000 results per call
10. **Python stdout buffering in containers** — use `PYTHONUNBUFFERED=1` or `flush=True`

📚 STUDY PLAN RECOMMENDATION

Week 1 — Foundation (Topics 1-8)

Practice: Write a log parser that reads a log file, groups entries by level, and writes a JSON summary

Week 2 — System Automation (Topics 9-15)

Practice: Write a config management tool that loads YAML configs with env var overrides and validates with Pydantic

Week 3 — API and Concurrency (Topics 16-18)

Practice: Write a concurrent health checker for 50+ endpoints with Prometheus metrics

Week 4 — Advanced Python (Topics 19-24)

Practice: Write a production-ready deployment library with retry decorator, circuit breaker, and full test coverage

Week 5 — Cloud & Infra (Topics 25-34)

Practice: Write a full deployment script: build → push → ECS/EKS deploy → health check → alert on failure

---

Generated by Claude — Anthropic | Version: DevOps/SRE Python Interview Guide v1.0 For each topic: Quiz Me | Go Deeper | Next Topic